



UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Calculatoare



Rețele Petri Stohastice - Model de simulare pentru evaluarea performanțelor în sisteme de servire cu topologie complexă

PROIECT DE DIPLOMĂ

Coordonator științific:
Prof.dr.ing. Petru CAȘCAVAL

Absolvent:
Emanuela CUCINSCHI

Iași, 2026

Cuprins

1	Introducere	1
1.1	Contextul temei și motivația	1
1.2	Obiectivele lucrării	1
1.3	Structura lucrării	2
1.4	Metodologia utilizată	2
2	Fundamentarea teoretică și soluții similare	3
2.1	Fundamentare teoretică	3
2.1.1	Sisteme cu evenimente discrete	3
2.1.2	Rețele Petri	3
2.1.3	Rețele Petri stohastice	4
2.1.4	Sisteme de servire și indicatori de performanță	5
2.2	Modele și soluții existente	5
2.3	Limitări ale soluțiilor existente și cerințele modelului propus	5
3	Soluția propusă	7
3.1	Formularea problemei și strategia de abordare	7
3.2	Arhitectura sistemului	8
3.3	Algoritmi și Logica de simulare	11
3.4	Alegerea tehnologiei	12
3.5	Descrierea implementării și Pseudocod	12
4	Testarea soluției și rezultate experimentale	15
4.1	Metodologia de testare	15
4.2	Modelul în serie	16
4.3	Modelul paralel	18
4.4	Modelul sincron	19
4.5	Sinteza modelelor de bază și impactul variabilității	21
4.6	Modelul compus	24
5	Concluzii	29
5.1	Direcții viitoare de dezvoltare	30
5.2	Lecții învățate pe parcursul dezvoltării proiectului de diplomă	30
	Bibliografie	31
	Anexe	33
1	Generatoare de variabile aleatoare	33
2	Modelul serie	33
3	Modelul paralel	34
4	Modelul sincron	35
5	Modelul compus	36

Rețele Petri Stohastice

Model de simulare pentru evaluarea performanțelor în sisteme de servire cu topologie complexă

Emanuela CUCINSCHI

Rezumat

Această lucrare abordează evaluarea performanțelor în sistemele de servire cu topologie complexă, utilizând formalismul rețelelor Petri stohastice. Această abordare depășește limitările modelelor analitice bazate strict pe teoria cozilor de așteptare, oferind un cadru robust pentru modelarea sistemelor moderne caracterizate prin procese concurente, resurse partajate și mecanisme stricte de sincronizare.

Din punct de vedere metodologic, a fost proiectat un model capabil să integreze mecanisme multiple de dirijare a traficului: cozi cu servire comună, reguli de prioritate nepreemptivă și etape de procesare paralelă cu sincronizare. Pentru evaluarea sa cantitativă, a fost implementat un simulator cu evenimente discrete în limbajul C++. Soluția dezvoltată utilizează distribuții probabilistice (exponențială și normală) pentru a modela timpii de sosire și de servire, permițând testarea rețelei sub diferite grade de variabilitate.

Analiza s-a concentrat pe evaluarea timpului mediu de răspuns și a lungimii maxime a cozilor de așteptare. Rezultatele simulărilor confirmă că performanța sistemului depinde nu doar de arhitectură, ci și de variabilitatea timpilor de prelucrare. Concret, s-a demonstrat că reducerea dispersiei timpilor de servire diminuează semnificativ timpii de așteptare și previne apariția blocajelor.

În concluzie, lucrarea arată că simularea este o metodă excelentă de a observa funcționalitatea unui sistem și de a asigura o gestionare eficientă a resurselor, înainte de a implementa sistemul în practică.

Cuvinte-cheie: rețele Petri stohastice, simulare cu evenimente discrete, sisteme de servire, prioritate nepreemptivă, sincronizare fork-join, teoria cozilor de așteptare, variabilitatea timpilor de servire.

Capitolul 1. Introducere

Sistemele de servire cu topologie complexă apar frecvent în domenii precum sistemele de calcul paralele, rețelele de comunicații, sistemele distribuite și aplicațiile în care mai multe cereri folosesc resurse comune. În aceste sisteme, evaluarea performanțelor este importantă încă din faza de proiectare, deoarece oferă posibilitatea de a analiza comportamentul și evoluția sistemului în diferite condiții, înainte de implementare sau utilizare practică. Indicatorii utilizați pentru evaluarea performanței, precum timpul mediu de răspuns, lungimea cozilor de așteptare și gradul de utilizare a resurselor, oferă informații esențiale despre eficiența și stabilitatea sistemului analizat.

Tema lucrării propune dezvoltarea unui simulator software bazat pe Rețele Petri Stohastice, utilizat pentru evaluarea performanțelor în sisteme de servire cu topologii diferite. Sunt analizate modele de organizare în serie, în paralel și modele cu sincronizare, urmărindu-se modul în care structura sistemului și distribuția timpilor de servire influențează timpul mediu de răspuns și lungimea cozilor de așteptare.

1.1. Contextul temei și motivația

În prezent, multe sisteme reale sunt formate din procese care se desfășoară în paralel și care implică numeroase elemente ce interacționează între ele, precum: resurse, date, procese și reguli de funcționare. Astfel de sisteme pot fi întâlnite în domenii precum transportul, comunicațiile, producția, serviciile publice sau sistemele informatice.

Deoarece un sistem real este permanent dependent de elementele din care este format și de factorii externi care influențează funcționarea acestuia, devine esențială identificarea și evaluarea unor metode capabile să îmbunătățească performanța acestor sisteme, vizând direct minimizarea timpilor de așteptare, gestionarea optimă a resurselor și sporirea vitezei de execuție.

Pentru studiul unor astfel de arhitecturi complexe, metodele analitice clasice devin limitate, în special atunci când intervin mecanisme de concurență sau partajare a resurselor. Din acest motiv, este necesară o abordare de modelare capabilă să descrie clar dinamica proceselor. În acest sens, formalismul Rețelelor Petri reprezintă un instrument optim, capabil să stea la baza unor simulatoare software robuste [1], deoarece nu surprinde doar stările finale, ci evidențiază cu exactitate regulile și condițiile care determină evoluția întregului sistem.

Mai mult, pentru a reflecta fidel realitatea (în care apar frecvent cozi de așteptare, utilizare neuniformă a resurselor și timpi variabili de execuție), se recurge la Rețelele Petri Stohastice. Acestea extind modelul de bază prin introducerea factorului probabilistic, reușind astfel să surprindă mult mai fidel funcționarea sistemelor reale și să valideze corect soluțiile de optimizare.

1.2. Obiectivele lucrării

Această lucrare urmărește utilizarea Rețelelor Petri Stohastice pentru simularea și evaluarea performanțelor unor sisteme de servire cu topologie complexă.

Obiectivul general al cercetării este dezvoltarea unui mediu de simulare care să permită modelarea, testarea și compararea mai multor tipuri de sisteme de servire. Pentru atingerea acestui scop, obiectivele specifice urmărite sunt:

- construirea unor modele care descriu cum sunt procesate cererile clienților într-un sistem;
- implementarea unui simulator cu evenimente discrete în limbajul C++;
- modelarea timpilor de sosire și de servire prin variații probabilistice;

- analizarea comportamentului sistemelor organizate în serie, paralel și sincron;
- compararea rezultatelor obținute pentru distribuția exponențială și distribuția normală a fiecărui tip de sistem analizat;
- identificarea influenței dispersiei timpilor de prelucrare asupra performanțelor sistemului;
- analizarea performanței sistemului prin urmărirea timpilor de așteptare, a utilizării resurselor și a posibilelor blocaje apărute în timpul simulării.

1.3. Structura lucrării

Prezenta lucrare este organizată în cinci capitole, structurate astfel încât să asigure o tranziție logică de la conceptele teoretice la validarea experimentală a soluției propuse:

Capitolul 1 prezintă introducerea, contextul temei, motivația alegerii acestuia, obiectivele urmărite prin prezenta cercetare și structura generală a lucrării.

Capitolul 2 conține fundamentarea teoretică necesară înțelegerii modelelor dezvoltate, abordând concepte de bază privind sistemele de servire, precum și teoria Rețelelor Petri și a Rețelelor Petri Stohastice.

Capitolul 3 detaliază soluția propusă, descriind componentele sistemului, arhitectura modelelor (serie, paralel, sincron) și modul în care acestea sunt reprezentate în cadrul rețelei Petri stohastice.

Capitolul 4 este dedicat simulării modelului și prezentării rezultatelor experimentale, incluzând metodologia de testare și analiza indicatorilor de performanță obținuți.

Capitolul 5 sintetizează concluziile finale desprinse în urma analizei rezultatelor și propune posibile direcții de dezvoltare viitoare ale aplicației și modelelor studiate.

1.4. Metodologia utilizată

Metodologia utilizată în această lucrare se bazează pe simularea cu evenimente discrete, o tehnică adecvată pentru modelarea sistemelor în care schimbările de stare se produc instantaneu, la momente specifice de timp (precum sosirea unei cereri sau finalizarea unei prelucrări).

Pentru a evalua performanța topologiilor propuse, metodologia de lucru a urmat un proces structurat în mai multe etape:

1. **Definirea sistemelor de servire:** S-au identificat și selectat arhitecturile de interes (modelele în serie, paralel și sincron), stabilindu-se componentele esențiale ale acestora și modul de interacțiune.
2. **Modelarea formală:** Sistemele identificate au fost transpuse în modele matematice riguroase utilizând formalismul rețelelor Petri, prin definirea precisă a locurilor, tranzițiilor, arcelor de conectare și a distribuției inițiale a jetoanelor.
3. **Integrarea factorului temporal:** Pentru a reflecta comportamentul real, modelelor le-au fost atașați timpi stohastici de execuție asociați tranzițiilor, transformându-le astfel în rețele Petri temporizate.
4. **Implementarea și execuția simulărilor:** Modelele au fost rulate în cadrul simulatorului software dezvoltat, parametrii de intrare (precum ratele de sosire și de servire) fiind variați controlat pentru a forța sistemele să tranziteze de la regimul stabil către pragul de saturație.
5. **Analiza și interpretarea datelor:** Ultima etapă a constat în colectarea indicatorilor (timpul mediu de răspuns, dimensiunea maximă a cozilor) și analiza lor comparativă, pentru a formula concluzii fundamentate privind eficiența fiecărei topologii.

Capitolul 2. Fundamentarea teoretică și soluții similare

Rețelele Petri reprezintă un model matematic puternic pentru descrierea și simularea sistemelor cu evenimente discrete. Ele nu exclud metodele clasice de analiză, ci le completează, facilitând înțelegerea comportamentului dinamic al sistemelor complexe. Acest instrument devine esențial atunci când arhitectura analizată implică procese concurente, sincronizare între resurse și partajarea acestora, contexte în care declanșarea evenimentelor prezintă un caracter nedeterminist. Înainte de a discuta soluțiile existente din literatură și limitările lor, este necesară o clarificare a conceptelor pe care se sprijină această abordare.

2.1. Fundamentare teoretică

Această secțiune introduce, pe rând, noțiunea de sistem cu evenimente discrete, formalismul rețelilor Petri și al extensiei lor stohastice, precum și elementele de bază din teoria sistemelor de servire, concepte pe baza cărora sunt discutate, în secțiunile următoare ale capitolului, soluțiile existente și limitările acestora.

2.1.1 Sisteme cu evenimente discrete

Un sistem cu evenimente discrete se caracterizează printr-o stare care nu evoluează continuu în timp, ci se modifică punctual, prin salturi instantanee, exclusiv la apariția unor evenimente bine definite. Spre deosebire de sistemele continue, modelate prin ecuații diferențiale, în cazul sistemelor cu evenimente discrete starea rămâne constantă între două tranziții succesive (de exemplu, nu există stări intermediare de „procesare parțială”).

Într-o arhitectură de servire, evenimentele discrete tipice includ sosirea unei cereri, inițierea procesării, finalizarea servirii sau eliberarea unei resurse partajate.

Pentru a înțelege complet comportamentul acestor arhitecturi, este esențială distincția dintre modul în care se schimbă starea sistemului și momentul declanșării acestor schimbări. Conceptul de „eveniment discret” definește exclusiv *cum* se modifică starea. În schimb, natura deterministă sau stohastică a proceselor definește *când* au loc aceste evenimente. Dacă momentele de apariție ale evenimentelor ar fi cunoscute cu certitudine absolută (de exemplu, sosiri la intervale fixe de timp), sistemul ar fi unul determinist.

În practică, însă, o asemenea certitudine este aproape imposibil de atins. Timpii de sosire și duratele de execuție fluctuează, fiind dependenți de o multitudine de factori, ceea ce transformă ansamblul într-un sistem *stochastic*. Prin urmare, arhitecturile studiate în această lucrare se încadrează simultan în ambele categorii: sunt cu evenimente discrete (deoarece stările se modifică prin tranziții punctuale) și stohastice (deoarece momentele declanșării evenimentelor sunt generate pe baza unor variabile aleatoare).

Evaluarea riguroasă a performanțelor unor astfel de sisteme complexe impune utilizarea simulatoarelor cu evenimente discrete și a unor metode probabilistice de analiză matematică, instrumente ce vor fi detaliate în secțiunile următoare ale acestui capitol.

2.1.2 Rețele Petri

O rețea Petri este, în forma sa cea mai simplă, un graf orientat format din două categorii de noduri: locațiile și tranzițiile. Între acestea sunt permise arce doar de la un tip de nod la celălalt, niciodată între două noduri de același tip [2].

În ceea ce privește comportamentul rețelei, declanșarea unei tranziții validate presupune eliminarea jetoanelor din locațiile de intrare și adăugarea lor în locațiile de ieșire, reproducând astfel trecerea sistemului dintr-o stare în alta. Pentru a ilustra acest concept, într-o arhitectură de servire cu o singură stație, o locație poate reprezenta coada de așteptare, în care fiecare jeton corespunde unei cereri. Tranziția asociată începerii servirii devine executabilă exclusiv în momentul în care există cel puțin un jeton disponibil în acea locație.

Din punct de vedere matematic, o rețea Petri netemporizată reprezintă un cvintuplu $PN = (P, T, I, O, M_0)$, având următoarele componente:

- P este mulțimea finită a locațiilor;
- T este mulțimea finită a tranzițiilor, cu condiția ca $P \cap T = \emptyset$ (mulțimi disjuncte);
- $I : P \times T \rightarrow \{0, 1, 2, \dots\}$ reprezintă funcția de incidență înainte, care definește ponderea arcelor orientate de la locații către tranziții;
- $O : T \times P \rightarrow \{0, 1, 2, \dots\}$ reprezintă funcția de incidență înapoi, care definește ponderea arcelor orientate de la tranziții către locații;
- $M_0 : P \rightarrow \{0, 1, 2, \dots\}$ este marcajul inițial al rețelei.

Dinamica rețelei este determinată de regula tranziției. O tranziție $t \in T$ este considerată validată într-un anumit marcaj M dacă și numai dacă $M(p) \geq I(p, t)$ pentru orice locație de intrare $p \in P$. Executarea unei tranziții validate generează un nou marcaj M' , calculat conform ecuației de stare:

$$M'(p) = M(p) - I(p, t) + O(t, p), \quad \forall p \in P$$

Această regulă fundamentală a validării și executării tranzițiilor stă la baza mecanismului de antrenare a rețelei, constituind nucleul algoritmilor de simulare detaliați în Capitolul 3.

2.1.3 Rețele Petri stohastice

Modelul de bază al rețelelor Petri surprinde doar ordinea posibilă de producere a evenimentelor, fără nicio informație despre durata lor. Pentru a putea evalua cantitativ un sistem, nu doar dacă o anumită secvență de evenimente este posibilă, ci și cât timp durează, este necesară o extensie temporizată a modelului, în care tranzițiilor li se asociază durate.

Când aceste durate nu sunt constante, ci variabile aleatoare rețeaua este numită stohastică [3]. Această variabilitate este, de altfel, o proprietate caracteristică a sistemelor reale de servire: doi clienți succesivi rareori necesită exact același timp de procesare, iar un model care ar ignora această variație ar simplifica excesiv comportamentul real al sistemului.

Alegerea distribuției exponențiale pentru modelarea duratelor aleatoare nu este întâmplătoare: pe lângă faptul că reflectă rezonabil variabilitatea proceselor de sosire și de servire din multe sisteme reale, proprietatea de lipsă de memorie a acestei distribuții simplifică considerabil analiza stărilor sistemului. Această proprietate este exploatată explicit în capitolele următoare, unde ratele de sosire și de servire ale sistemului analizat sunt eșantionate din distribuții exponențiale.

Pe lângă distribuția exponențială, utilă datorită proprietății de lipsă de memorie, în modelarea stohastică se utilizează frecvent și distribuția normală (gaussiană). Ea permite reprezentarea proceselor ale căror durate sunt simetrice față de o valoare medie, fiind utilă în analizarea impactului pe care dispersia și fluctuațiile controlate ale timpilor de servire le au asupra performanței generale. În cadrul simulatorului dezvoltat pentru această lucrare, ambele modele de distribuție au fost implementate și evaluate comparativ, urmărindu-se modul în care comportamentul stohastic influențează indicatorii de performanță ai sistemului.

2.1.4 Sisteme de servire și indicatori de performanță

Un sistem de servire este caracterizat, în esență, de un flux de cereri care sosesc conform unui anumit tipar, de una sau mai multe stații care le prelucrează, de o disciplină de servire (de exemplu, principiul primul sosit, primul servit — FIFO) și de regulile după care cererile părăsesc sistemul odată procesate.

Pentru a evalua performanța unui asemenea sistem se urmăresc, de regulă, câțiva indicatori standard: timpul mediu de așteptare, timpul mediu total petrecut de o cerere în sistem, lungimea medie și maximă a cozilor de așteptare, gradul de utilizare a fiecărei stații de servire și numărul de cereri procesate într-un interval dat. Aceștia constituie setul principal de parametri analizați în capitolul de rezultate experimentale al prezentei lucrări, pentru fiecare dintre topologiile evaluate.

O notație consacrată pentru descrierea succintă a unui sistem de servire este cea introdusă de Kendall [4], de forma $A/B/c$, unde A descrie procesul de sosire, B procesul de servire, iar c numărul de servere.

2.2. Modele și soluții existente

Pentru analiza sistemelor cu evenimente discrete există, în literatura de specialitate, mai multe abordări consacrate.

Modelele Markov oferă un cadru analitic riguros, bazat pe graful stărilor sistemului și pe ratele de tranziție dintre acestea. Problema principală apare atunci când sistemul modelat are mai multe componente care pot evolua independent, caz în care numărul stărilor posibile crește exponențial, fenomen cunoscut ca explozia spațiului de stări. Teoria clasică a cozilor de așteptare (notația Kendall, de exemplu $M/M/1$ sau $M/M/c$) oferă formule analitice pentru sisteme cu o structură simplă, dar devine greu de aplicat pentru topologii cu mai multe etape, resurse partajate sau reguli de prioritate.

Rețelele Petri stohastice oferă un compromis util între cele două abordări. Spre deosebire de un lanț Markov construit direct, o rețea Petri păstrează o reprezentare grafică explicită a structurii sistemului care facilitează documentarea și identificarea vizuală a resurselor critice, permițând, în același timp, o evaluare cantitativă riguroasă.

2.3. Limitări ale soluțiilor existente și cerințele modelului propus

Rigurozitatea matematică a abordărilor analitice standard este adesea contrabalansată de o lipsă de scalabilitate și flexibilitate în fața arhitecturilor complexe. Principalul impediment este ilustrat de modelele Markov, a căror complexitate de calcul crește exponențial în prezența mecanismelor concurente, generând imposibilitatea evaluării practice din cauza exploziei spațiului de stări.

Teoria cozilor de așteptare oferă rezultate analitice exacte doar pentru topologii relativ simple, majoritatea rezultatelor clasice presupunând o singură stație sau structuri foarte regulate, ceea ce le face greu de extins la sisteme cu prioritate între clase de cereri sau cu sincronizare între mai multe resurse. Această dificultate este confirmată explicit în literatură: chiar și pentru cea mai simplă rețea de sincronizare fork-join, cu doar două stații, soluția exactă a necesitat funcții generatoare complexe [5], iar pentru cazuri mai generale nu există, până în prezent, decât metode aproximative [6]. În mod similar, analiza cozilor cu prioritate nepreemptivă și mai multe servere rămâne, în general, intratabilă analitic dincolo de cazul particular al a două clase de prioritate cu timpi de servire exponențiali [7].

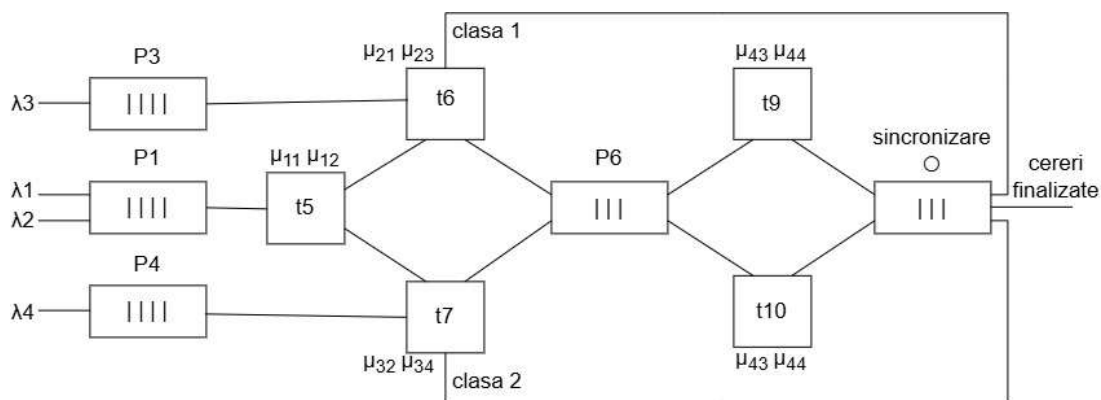
În plus, niciuna dintre cele două abordări nu oferă o reprezentare grafică primară a sistemului, utilă atât pentru comunicarea arhitecturii, cât și pentru identificarea intuitivă a blocajelor. Pornind de la aceste limitări, modelul propus trebuie să răspundă unui set clar de cerințe structurale și funcționale. În primul rând, este necesară reprezentarea unui sistem de servire cu mai multe etape de prelucrare și clase distincte de cereri, modelând explicit sosirea entităților, formarea cozilor de așteptare și gradul de disponibilitate a resurselor. În al doilea rând, sistemul trebuie să permită alocarea unor timpi de sosire și de servire variabili, definiți sub forma unor variabile aleatoare. De asemenea, arhitectura

trebuie să integreze o regulă de prioritate clară între clasele de cereri care concurează pentru aceeași stație de servire. Nu în ultimul rând, modelul trebuie să permită reproducerea fidelă a unei etape de sincronizare, în care finalizarea unui proces depinde de terminarea simultană a activităților pe mai multe ramuri paralele.

Capitolul 3 detaliază modul concret în care aceste cerințe au fost transpuse într-o arhitectură de rețea Petri stohastică, precum și algoritmul de simulare folosit pentru evaluarea acesteia.

3.1. Formularea problemei și strategia de abordare

Pentru a aborda aceste provocări într-un cadru controlat, lucrarea analizează un sistem de servire capabil să încapsuleze exact dificultățile de alocare și sincronizare menționate. Sistemul studiat (Figura 3.1) este structurat pe patru clase de cereri (λ_1 – λ_4). Interacțiunile și potențialele blocaje sunt modelate direct: cererile din clasele 1 și 2 concurează pentru o stație comună, în timp ce clasele 3 și 4 dispun de cozi de așteptare dedicate. Ca soluție de gestionare a fluxului, se aplică o politică de prioritate nepreemptivă care favorizează clasele 1 și 2, finalizându-le imediat după servire. Prin contrast, complexitatea sincronizării resurselor este abordată în prelucrarea claselor inferioare (3 și 4), care impun o etapă suplimentară de procesare concurentă, necesitând reunirea rezultatelor de la două unități paralele.



Această structură, care constă în cozi de dimensiune variabilă, regim de prioritate între clase și o etapă de sincronizare a două resurse, reprezintă exact tipul de complexitate care justifică alegerea unui formalism capabil să surprindă atât concurența, cât și aleatorietatea temporală a sistemului.

7

consacrat, capabil să surprindă riguros comportamentul logic și structural al sistemelor cu evenimente discrete [8]. Prin extinderea modelului clasic cu timpi aleatori de execuție, varianta stohastică permite o evaluare cantitativă detaliată a performanțelor sistemului, dincolo de simpla validare structurală [3].

Strategia de cercetare și implementare urmărește trei etape logice:

1. **Modelarea arhitecturală:** Proiectarea a trei topologii fundamentale (modelul cu organizare în serie, modelul paralel și modelul sincron). Acestea constituie structurile de bază pentru dezvoltarea, validarea și integrarea modelului complex de servire studiat, capabil să gestioneze cele patru clase de cereri concurente.
2. **Implementarea software:** Transpunerea modelelor teoretice într-un simulator cu evenimente discrete implementat în limbajul C++. Modelul de simulare utilizează generatoare de numere pseudo-aleatoare pentru a asocia tranzițiilor timpi de întârziere bazați pe distribuții probabilistice exponențiale (caracterizate de ratele de sosire λ și de servire μ).
3. **Analiza performanțelor:** Executarea simulărilor pe durate extinse pentru a colecta și interpreta metrice critice ale sistemului global, cum ar fi timpul mediu de răspuns și dimensiunea maximă a cozilor de așteptare, determinând astfel condițiile de stabilitate atât pentru modulele individuale, cât și pentru arhitectura integrată.

3.2. Arhitectura sistemului

Pentru a putea simula și analiza performanțele, procesele reale sunt modelate folosind arhitectura specifică Rețelelor Petri stohastice. Elementele fizice și logice ale sistemelor de servire au fost transpuse în componentele fundamentale ale acestor structuri grafice, astfel:

- **Locațiile și Jetoanele:** Locațiile modelează stările posibile ale sistemului (ex. cozi de așteptare, buffere de sincronizare sau starea de disponibilitate a unui server), iar marcajul curent (distribuția jetoanelor pe locații) indică în orice moment care dintre aceste stări este activă. Jetoanele reprezintă entitățile dinamice (sarcini, cereri) care traversează sistemul [8, 2].
- **Tranzițiile:** Modelează evenimentele discrete propriu-zise; executarea unei tranziții validate produce o nouă distribuție a jetoanelor, echivalentă cu o schimbare de stare a sistemului real [8, 2]. Pentru a putea cuantifica performanțele, s-a folosit extinderea temporizată a formalismului, în care fiecărei tranziții relevante i se atașează o durată — timpul care trebuie să treacă din momentul validării până la executarea ei efectivă [3]. Întrucât aceste durate sunt eșantionate din distribuții probabilistice (parametrizate prin λ pentru sosiri și μ pentru servire), rezultă o Rețea Petri stohastică [3].

Pentru a acoperi principalele moduri de interacțiune din sistemele concurente, arhitectura propusă analizează trei modele topologice distincte:

1. Modelul cu organizare în serie

Din punct de vedere structural, modelul (ilustrat în Figura 3.2) poate fi descris ca o înlănțuire a două subsisteme de servire cu o singură stație fiecare, fără nicio restricție asupra numărului de cereri aflate simultan în sistem [9]. Fluxul de procesare este strict secvențial: o cerere nu este considerată finalizată la primul nod decât în momentul în care devine disponibilă pentru al doilea, cele două evenimente fiind, de fapt, unul singur în model [9]. La nivel de rețea, un jeton parcurge succesiv locațiile și tranzițiile; intervalele dintre sosiri și duratele celor două etape de procesare sunt generate ca variabile aleatoare exponențiale, de parametri λ , respectiv μ_1 și μ_2 [10, 9].

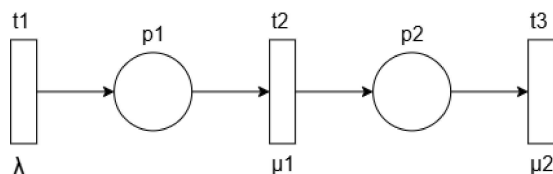


Figura 3.2: Modelul serie al rețelei Petri

2. Modelul cu organizare paralelă

Spre deosebire de organizarea în serie, aici cele două procesoare (Figura 3.3) nu mai depind unul de celălalt în timpul prelucrării: fiecare primește, în același moment, câte o componentă a aceleiași cereri pereche și o tratează independent, în ritm propriu [9]. Această independență introduce însă o problemă de sincronizare la final — cererea nu poate fi considerată încheiată decât după ce ambele componente au fost prelucrate. Din acest motiv, starea rețelei nu mai poate fi redusă la o singură coadă comună, ca la modelul serie; sunt necesare locații separate care să rețină, pe de o parte, cererile aflate încă în lucru la fiecare procesor, iar pe de altă parte, rezultatele parțiale deja obținute și care așteaptă perechea [2, 9]. Momentul de reunire este modelat printr-o tranziție fără timp asociat (o tranziție *receptor*): ea devine executabilă numai când în ambele locații de rezultate parțiale există câte un jeton, iar executarea ei elimină simultan cererea din sistem [3, 9].

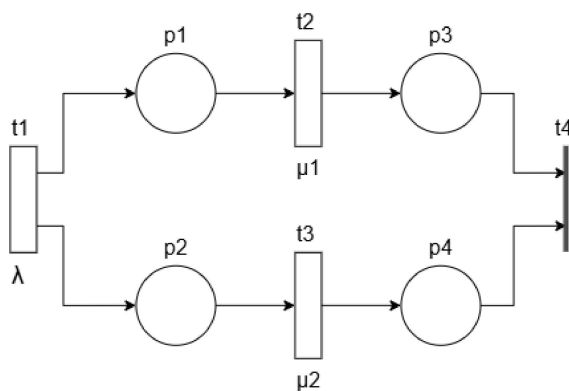


Figura 3.3: Modelul paralel al rețelei Petri

3. Modelul cu organizare sincronă

Modelul sincron (Figura 3.4) introduce o constrângere suplimentară față de cel paralel: cele două procesoare nu mai lucrează pe cereri diferite, ci trebuie să fie disponibile *simultan* pentru a începe o nouă prelucrare. Starea sistemului trebuie așadar să surprindă două lucruri distincte: câte cereri așteaptă să fie preluate și dacă procesoarele sunt, chiar în acest moment, libere amândouă [9]. Primul aspect e reprezentat printr-o locație obișnuită de coadă, fără jetoane la pornire, întrucât sistemul începe fără cereri în așteptare [2]. Al doilea aspect are nevoie de o locație separată, care pornește cu un jeton (semnalând că, inițial, resursele sunt libere) și care se golește exact atunci când o prelucrare începe [8, 2]. Pornirea unei prelucrări este, la rândul ei, o singură tranziție care mută cererea simultan în bufferele ambelor procesoare; abia când amândouă și-au terminat sarcina, cererea este scoasă din sistem și jetonul „resurse libere” revine în locația lui, redeschizând ciclul pentru următoarea cerere [8, 9].

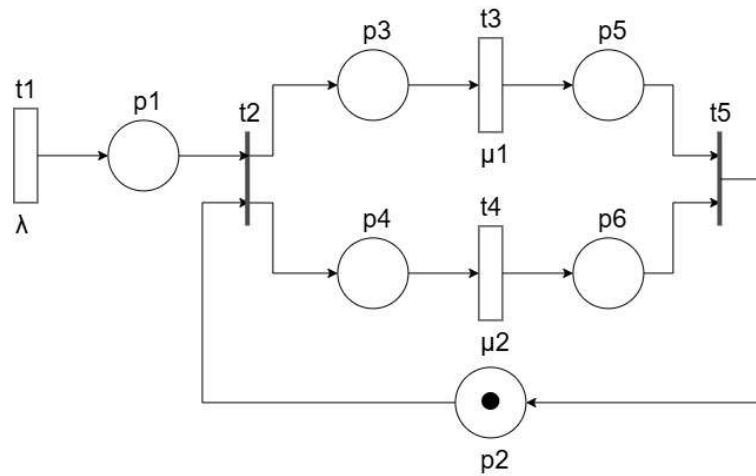


Figura 3.4: Modelul sincron al rețelei Petri

Deși cele trei arhitecturi prezentate surprind, fiecare, câte un mecanism fundamental de interacțiune între procese concurente (secvențialitate, paralelism, respectiv sincronizare), sistemele reale de servire combină adesea mai multe astfel de mecanisme simultan, iar cererile tratate nu sunt, de regulă, omogene ca tip. Contribuția originală a prezentei lucrări constă în proiectarea unui model compus, care extinde topologiile de bază prin integrarea următoarelor elemente arhitecturale:

1. Definirea a patru clase independente de cereri, cu intensități de sosire proprii (λ_1 – λ_4), grupate funcțional în două perechi pe baza traseului urmat în rețea;
2. Prima pereche (clasele 1 și 2) partajează o **stație comună de servire FIFO**, fiind tratată strict în ordinea sosirii, dar cu o rată de procesare dependentă de tipul cererii;
3. Fluxul rezultat concurează apoi cu a doua pereche (clasele 3 și 4, care așteaptă direct în cozi dedicate) într-o etapă cu **prioritate nepreemptivă**. Cele două servere alocate aici procesează prioritar cererile provenite din prima stație;
4. Sarcinile neprioritare (clasele 3 și 4) sunt direcționate final către o **etapă de sincronizare**. Finalizarea acestora impune ocuparea simultană a două ramuri de calcul, conform topologiei modelului sincron descris anterior.

Modelul rezultat (prezentat în Figura 3.5) permite observarea directă a modului în care cele trei mecanisme distincte (servirea FIFO cu rată dependentă de clasă, prioritatea nepreemptivă și sincronizarea de tip fork-join) se cumulează, influențând timpul de răspuns pentru diferitele clase de cereri, aspect ce va fi analizat cantitativ în Capitolul 4.

Structura rețelei este organizată în trei etape. În prima etapă, cererile de culoare 1 și 2 sosesc într-o locație comună, servită în ordinea sosirii de o singură tranziție temporizată; rezultatul este direcționat, în funcție de clasa cererii tocmai servite, spre una din două locații de așteptare distincte. Cererile de culoare 3 și 4 nu parcurg această primă etapă, sosind direct în locațiile lor proprii de așteptare. În a doua etapă, fiecare din cele două servere primește cereri din două surse: una cu prioritate (provenită din prima etapă) și una secundară, sosită direct. Locațiile asociate cererilor cu prioritate sunt conectate direct la tranziția de servire a serverului respectiv, iar cele secundare intră în competiție pentru același server, fiind selectate doar în absența unei cereri prioritare în așteptare. La finalizarea acestei a doua servări, cererile prioritare părăsesc sistemul (timpul lor de răspuns fiind calculat de la momentul sosirii inițiale, incluzând așadar durata petrecută în ambele etape de servire), în timp ce cererile secundare sunt direcționate spre locația comună de așteptare a celei de-a treia etape.

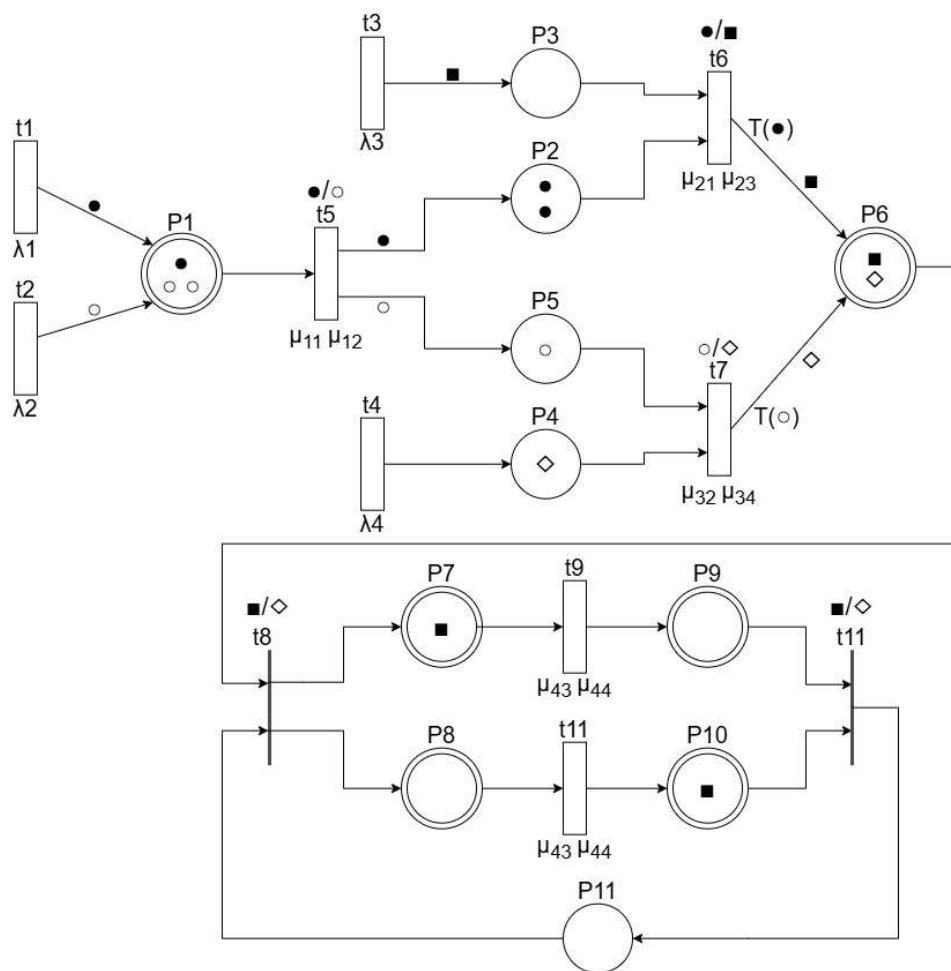


Figura 3.5: Modelul compus al rețelei Petri

3.3. Algoritmi și Logica de simulare

Un simulator cu evenimente discrete evaluează continuu evoluția stării sistemului pe axa timpului. Pentru majoritatea tranzițiilor din rețea, mecanismul este standard: fiecareia îi este asociată o durată (eșantionată la momentul validării), iar algoritmul identifică, la fiecare pas, tranziția a cărei durată rămasă este minimă, executând-o prima din punct de vedere cronologic. Totuși, evenimentele logice de dirijare, precum pornirea și încheierea etapei de sincronizare (Secțiunea 3.2), reprezintă *tranziții imediate* (ele nu consumă timp de simulare, ci se declanșează instantaneu la îndeplinirea condițiilor de validare).

Pentru a gestiona corect această dualitate temporală, bucla principală a programului a fost organizată în două faze distincte, executate la fiecare iterație:

- **Rezolvarea evenimentelor instantanee** — constă în execuția tuturor tranzițiilor logice (fără durată), precum ramificarea sau unificarea etapei de sincronizare. Tot în această fază se eșantionează și duratele noilor tranziții temporizate proaspăt validate și se aplică regulile de selecție pentru resursele partajate (servirea comună FIFO sau extragerea cu prioritate nepreemptivă), pregătind astfel starea sistemului pentru pasul următor;
- **Avansarea ceasului de simulare** — dintre toate tranzițiile temporizate aflate „în lucru” (sosirile cererilor, cele trei servicii de prelucrare și execuția efectivă a ramurilor de sincronizare), se selectează evenimentul cu timpul de așteptare minim, iar ceasul global avansează direct până la acel moment.

3.4. Alegerea tehnologiei

Implementarea simulatorului a fost realizată în limbajul C++, fără utilizarea unor biblioteci externe de simulare. Alegerea este motivată de:

- Nevoia de control direct asupra buclei de evenimente discrete și a structurilor de date asociate marcatului rețelei - greu de obținut cu aceeași claritate folosind un framework de simulare de nivel înalt;
- Continuitatea cu formalismul și stilul de implementare folosite pentru modelele fundamentale prezentate în Capitolul 2 - arhitecturile serie, paralelă și sincronă au fost verificate separat, iar modelul propus a fost construit adăugând, pe rând, fiecare mecanism nou (servire comună, prioritate, sincronizare) peste acest fundament deja funcțional;
- Nevoia de a rula, în timp rezonabil, un număr mare de simulări independente pe durate extinse - ceva ce un limbaj interpretat ar fi îngreunat considerabil, dat fiind volumul de calcul implicat de generarea repetată a variabilelor aleatoare și de gestionarea cozilor.

3.5. Descrierea implementării și Pseudocod

Implementarea simulatorului a fost realizată în limbajul C++, fără utilizarea unor biblioteci externe (motivată în Secțiunea 3.4). Structura de date centrală este vectorul duratelor $D[]$, asociat tranzițiilor temporizate ale rețelei (Secțiunea 3.3), în care:

- $D[i] = -1$ — tranziția i nu este validată;
- $D[i] = 0$ — tranziția i este executabilă imediat;
- $D[i] > 0$ — timpul rămas până la executarea tranziției i .

Structurile de date utilizate în implementare reflectă fidel locațiile și tranzițiile modelului teoretic, fiind organizate logic pe cele trei secțiuni ale rețelei:

Etapa 1a (sosiri și prima servire)

Locația P_1 acționează ca o coadă FIFO comună claselor 1 și 2. În cod, aceasta este reprezentată prin vectorii $P1_s[]$ și $P1_c[]$, alături de contorul $nP1$, reținând astfel momentul sosirii și clasa fiecărei cereri, în ordinea reală de intrare. Tranziția asociată, t_5 , își păstrează starea curentă de execuție prin variabilele $D[5]$, $t5_c$ și $t5_s$, care rețin durata rămasă, clasa, respectiv momentul sosirii cererii procesate.

Etapa 1b (servirea cu prioritate nepreemptivă)

Sistemul utilizează două servere distincte. Serverul superior (t_6) extrage cereri prioritare din locația P_2 ($P2[]$, $nP2$) și abia apoi, în absența acestora, din coada de sosire directă P_3 ($P3[]$, $nP3$). Starea sa logică este menținută prin variabilele $D[6]$, $t6_c$ și $t6_s$. În mod similar, serverul inferior (t_7) prioritizează clasa 2 din locația P_5 ($P5[]$) în detrimentul clasei secundare 4 sosite în P_4 ($P4[]$), folosind structurile de stare $D[7]$, $t7_c$ și $t7_s$.

Etapa 2 (sincronizarea fork-join)

Această etapă impune structuri de date dedicate concurenței. Cererile așteaptă în locația P_6 ($P6_s[]$, $P6_c[]$) cât timp resursa de sincronizare este ocupată. Disponibilitatea acestei resurse (P_{11}) este dictată de indicatorul boolean $P11$ (1 = etapă liberă, 0 = ocupată). Când o sarcină intră în execuție pe ambele ramuri, ea ocupă locațiile P_7 , P_8 (modelate prin variabilele $P7P8$, $P7P8_c$, $P7P8_s$). Cele două servere paralele (t_9 , t_{10}) își contorizează duratele prin $D[9]$ și $D[10]$, iar finalizarea individuală a ramurilor (P_9 , P_{10}) este semnalizată prin indicatorii t_9 și t_{10} , permițând astfel unificarea fluxului.

Notății și variabile de stare:

- $D[i]$: durata de execuție rămasă pentru tranziția temporizată t_i ;
- $P_1, \dots, P_6$: locațiile care modelează cozile de așteptare ale rețelei (conform Secțiunii 3.2). Fiecare cerere din sistem este modelată ca un tuplu de forma (t, c) , unde t reprezintă momentul sosirii în sistem, iar $c \in \{1, 2, 3, 4\}$ indică clasa cererii;
- Funcțiile $\text{EXTRAGE}(Q)$ și $\text{ADAUGA}(Q, t, c)$: implementează comportamentul de tip FIFO, eliminând, respectiv inserând o cerere la extremitățile cozii Q ;
- t_5 : stația de prelucrare comună pentru clasele 1 și 2, caracterizată de ratele de servire exponențiale $\mu_{5,1}$ și $\mu_{5,2}$. Starea curentă a serverului este descrisă de perechea (c_{t5}, s_{t5}) ;
- t_6, t_7 : stațiile de prelucrare cu prioritate nepreemptivă. Stația t_6 procesează prioritar clasa 1 (față de clasa 3), folosind ratele $\mu_{6,1}$ și $\mu_{6,3}$, în timp ce stația t_7 favorizează clasa 2 (față de clasa 4), cu ratele $\mu_{7,2}$ și $\mu_{7,4}$;
- t_9, t_{10} : ramurile etapei de sincronizare finală, având rate de procesare identice $\mu_{9,10}$ dependente de clasa cererii. Indicatorul boolean P_{11} semnalează disponibilitatea etapei de sincronizare, în timp ce P_7P_8 marchează prezența activă a unei cereri în procesare.

Condiții inițiale:

cozi goale ($P_1, \dots, P_6 = \emptyset$), $D[i] = -1$ pentru $i \in \{1, \dots, 11\}$, $ceas = 0$,
 $P_{11} = \text{true}$, $P_7P_8 = \text{false}$, $t_9 = t_{10} = \text{false}$.

Algorithm 1 Bucla principală de simulare

```

1: function BUCLASIMULARE( $DS$ )
2:    $ceas \leftarrow 0$ ;  $D[i] \leftarrow -1$  pentru  $i = 1, \dots, 11$ ;  $D[1..4] \leftarrow \text{EXP}(\lambda[1..4])$ 
3:   while  $ceas < DS$  do
4:     RULEAZATRANZITIE
5:      $t_e \leftarrow \arg \min_i D[i]$  astfel încât  $D[i] \geq 0$ ;  $t_{min} \leftarrow D[t_e]$ 
6:     if niciun  $t_e$  then oprește
7:     end if
8:     if  $ceas + t_{min} > DS$  then  $t_{min} \leftarrow DS - ceas$ 
9:     end if
10:     $ceas \leftarrow ceas + t_{min}$ 
11:     $D[i] \leftarrow D[i] - t_{min}$  pentru toți  $i$  astfel încât  $D[i] \geq 0$ 
12:    if  $ceas \geq DS$  then oprește
13:    end if
14:    EXECUTA( $t_e$ )
15:  end while
16:  return  $STr[\ ]$ ,  $CT[\ ]$ 
17: end function

```

Algorithm 2 Procesarea tranzițiilor

```

1: function RULEAZATRANZITIE
2:   repeat
3:      $ok \leftarrow \text{false}$ 
4:     if  $D[5] < 0$  and  $|P1| > 0$  then
5:        $(t5\_c, t5\_s) \leftarrow \text{EXTRAGE}(P1)$ ;  $D[5] \leftarrow \text{EXP}(\mu_5(t5\_c))$ ;  $ok \leftarrow \text{true}$ 
6:     end if
7:     if  $D[6] < 0$  and ( $|P2| > 0$  or  $|P3| > 0$ ) then
8:        $(t6\_c, t6\_s) \leftarrow \text{EXTRAGE}(P2)$  dacă  $|P2| > 0$ , altfel  $\text{EXTRAGE}(P3)$ 

```

```

9:       $D[6] \leftarrow \text{Exp}(\mu_6(t6\_c)); ok \leftarrow \text{true}$ 
10:    end if
11:    if  $D[7] < 0$  and  $(|P5| > 0 \text{ or } |P4| > 0)$  then
12:       $(t7\_c, t7\_s) \leftarrow \text{EXTRAGE}(P5)$  dacă  $|P5| > 0$ , altfel  $\text{EXTRAGE}(P4)$ 
13:       $D[7] \leftarrow \text{Exp}(\mu_7(t7\_c)); ok \leftarrow \text{true}$ 
14:    end if
15:    if  $\neg P7P8$  and  $P11$  and  $|P6| > 0$  then
16:       $(P7P8\_c, P7P8\_s) \leftarrow \text{EXTRAGE}(P6)$ 
17:       $P7P8 \leftarrow \text{true}; P11 \leftarrow \text{false}; t9, t10 \leftarrow \text{false}$ 
18:       $D[9] \leftarrow \text{Exp}(\mu_{9,10}(P7P8\_c)); D[10] \leftarrow \text{Exp}(\mu_{9,10}(P7P8\_c)); ok \leftarrow \text{true}$ 
19:    end if
20:    if  $P7P8$  and  $t9$  and  $t10$  then
21:       $\text{STr}[P7P8\_c] \leftarrow \text{STr}[P7P8\_c] + (ceas - P7P8\_s); CT[P7P8\_c] \leftarrow$ 
 $CT[P7P8\_c] + 1$ 
22:       $P7P8 \leftarrow \text{false}; P11 \leftarrow \text{true}; ok \leftarrow \text{true}$ 
23:    end if
24:  until  $ok = \text{false}$ 
25: end function

```

Algorithm 3 Executarea tranziției stabilite

```

1: procedure EXECUTA( $t_e$ )
2:   if  $t_e \in \{1, 2\}$  then
3:      $\text{ADAUGA}(P1, ceas, t_e); D[t_e] \leftarrow \text{Exp}(\lambda[t_e])$ 
4:   else if  $t_e = 3$  then
5:      $\text{ADAUGA}(P3, ceas); D[3] \leftarrow \text{Exp}(\lambda[3])$ 
6:   else if  $t_e = 4$  then
7:      $\text{ADAUGA}(P4, ceas); D[4] \leftarrow \text{Exp}(\lambda[4])$ 
8:   else if  $t_e = 5$  then
9:      $\text{ADAUGA}(P2, t5\_s)$  dacă  $t5\_c = 1$ , altfel  $\text{ADAUGA}(P5, t5\_s); D[5] \leftarrow -1$ 
10:  else if  $t_e = 6$  then
11:    if  $t6\_c = 1$  then  $\text{STr}[1] \leftarrow \text{STr}[1] + (ceas - t6\_s); CT[1] \leftarrow CT[1] + 1$ 
12:    else  $\text{ADAUGA}(P6, t6\_s, 3)$ 
13:    end if
14:     $D[6] \leftarrow -1$ 
15:  else if  $t_e = 7$  then
16:    if  $t7\_c = 2$  then  $\text{STr}[2] \leftarrow \text{STr}[2] + (ceas - t7\_s); CT[2] \leftarrow CT[2] + 1$ 
17:    else  $\text{ADAUGA}(P6, t7\_s, 4)$ 
18:    end if
19:     $D[7] \leftarrow -1$ 
20:  else if  $t_e = 9$  then
21:     $t9 \leftarrow \text{true}; D[9] \leftarrow -1$ 
22:  else if  $t_e = 10$  then
23:     $t10 \leftarrow \text{true}; D[10] \leftarrow -1$ 
24:  end if
25: end procedure

```

Pentru cererile din clasele 1 și 2, momentul sosirii inițiale se propagă prin $t5_s$ și $P2/P5$, astfel încât timpul de răspuns ($ceas -$ sosire) acoperă ambele etape de servire. Deși teoretic cozile sunt infinite, implementarea C++ utilizează vectori de 100.000 de elemente, capacitate validată empiric ca fiind suficientă pentru a evita depășirile la ratele simulate.

Capitolul 4. Testarea soluției și rezultate experimentale

4.1. Metodologia de testare

Pentru fiecare dintre cele trei topologii analizate, modelul cu organizare în serie, modelul paralel și modelul sincron, evaluarea performanțelor s-a realizat prin variații controlate ale parametrilor de intrare ai simulării, urmărind două obiective: identificarea condițiilor de stabilitate ale sistemului și cuantificarea impactului variabilității timpilor de servire asupra timpului mediu de răspuns.

Parametrii variați sistematic au fost:

- **rata de sosire** λ , crescută progresiv spre pragul de saturație al sistemului, pentru a observa tranziția de la regim stabil la regim instabil;
- **ratele de servire** μ_1 și μ_2 ale celor două stații, testate în mai multe combinații (egale și inegale), pentru a evidenția influența dezechilibrului dintre stații asupra performanței globale;
- **durata simulării** DS , variată între 10^5 și 10^6 unități de timp, pentru a verifica convergența rezultatelor spre valori stabile în regim staționar.

Configurațiile testate formează un experiment controlat: un sistem simetric ($\mu_1 = \mu_2 = 10$), un dezechilibru major ($\mu_1 = 11$, $\mu_2 = 9$) și o asimetrie minoră ($\mu_1 = 11$, $\mu_2 = 10$). Valorile parametrului λ au fost alese diferit pentru modelul sincron față de modelele serie și paralel, pentru a ține cont de diferențele apărute la pragul de saturație.

Pentru evaluarea fiecărei configurații s-au monitorizat trei indicatori: timpul mediu de răspuns (Trm) și dimensiunea maximă atinsă de cozile de așteptare ($maxP1$, $maxP2$). Analiza nu s-a limitat la performanța generală indicată de medii, deoarece un Trm moderat poate ascunde vârfurile extreme de trafic și momentele de blocaj temporar ale sistemului. Urmărirea dimensiunii maxime a cozilor este esențială, deoarece, în practică, resursele fizice ale unui sistem de calcul (precum memoria alocată bufferelor) trebuie dimensionate pentru a face față celui mai rău scenariu posibil (vârful maxim de cereri) și nu doar unei încărcări medii.

Pentru a evidenția faptul că performanța unui sistem de servire depinde nu doar de capacitatea medie de procesare, ci și de predictibilitatea acesteia, fiecare topologie a fost testată în două scenarii. Primul utilizează o distribuție exponențială a timpilor de servire, $Exp(\mu_i)$, caracterizată printr-o dispersie ridicată. Al doilea scenariu folosește o distribuție normală, $N(1/\mu_i, \sigma_i)$, care păstrează aceeași medie a timpului de execuție ($1/\mu_i$), dar limitează fluctuațiile prin fixarea abaterii standard la o valoare redusă, $\sigma_i = 1/(5\mu_i)$. Această constrângere a variațiilor haotice permite izolarea clară a impactului pe care îl are variabilitatea timpului de servire asupra stabilității globale a sistemului, aspect analizat pe larg în Secțiunea 4.5.

Toate simulările au fost rulate pornind de la un sistem fără cereri în așteptare, motiv pentru care, prima etapă a execuției nu reflectă performanța stabilă a sistemului. Pentru a preveni distorsionarea rezultatelor finale de către această perioadă de inițializare, experimentele au fost rulate pe durate foarte mari. În acest fel, impactul măsurătorilor de la început este minimizat, iar acuratețea datelor finale este demonstrată prin analiza de convergență prezentată la finalul fiecărei secțiuni următoare (Tabelele 4.3, 4.6, 4.9).

Valorile marcate cu asterisc * indică situațiile în care sistemul este forțat la sau peste pragul teoretic de stabilitate ($\rho \geq 1$). Deoarece serverele nu mai fac față ratei de sosire, sistemul nu atinge un regim staționar, iar cozile cresc la nesfârșit. Din acest motiv, rezultatele pentru Trm și $maxP$ nu reprezintă valori de echilibru, ci reflectă pur și simplu nivelul de congestie atins exact în momentul în care simularea a fost oprită.

4.2. Modelul în serie

În analiza modelului în serie, un aspect esențial îl reprezintă modul în care cererile parcurg cele două stații. Deoarece întregul flux de evenimente generat cu rata λ trebuie procesat secvențial, capacitatea de prelucrare a întregului sistem nu este o medie a celor două, ci este strict limitată de stația cu cea mai mică viteză de servire.

Prin urmare, condiția teoretică de stabilitate impune ca rata de sosire să fie strict mai mică decât rata de servire a acestei stații. Gradul de încărcare a sistemului, ρ , este determinat tocmai de această stație critică și se calculează astfel:

$$\rho = \frac{\lambda}{\min(\mu_1, \mu_2)}$$

Pentru ca modelul să atingă o stare staționară, este necesar ca $\rho < 1$. Dacă $\rho \geq 1$, sistemul devine instabil, fenomen caracterizat printr-o acumulare necontrolată a cererilor în coada stației mai lente.

Pentru a observa mai clar dinamica acestei configurații, scenariile de testare au fost alese astfel încât să acopere diferite niveluri de încărcare. Rulările pornesc de la un regim stabil ($\lambda = 7$, corespunzător unui grad de încărcare $\rho \in [0,70, 0,78]$) și avansează progresiv către punctul de saturație al fiecărei configurații. Valorile din zona critică ($\lambda \in \{9; 9,5; 9,7; 9,9; 10\}$) au fost alese cu un pas mai mic pentru a oferi o precizie mai bună în imediata vecinătate a pragului $\rho \rightarrow 1$. Această abordare ajută la evidențierea creșterii timpului mediu de răspuns (T_{rm}) și a lungimii cozilor la trecerea din regimul stabil în cel instabil.

	$\mu_1 = 11, \mu_2 = 9$			$\mu_1 = 10, \mu_2 = 10$			$\mu_1 = 11, \mu_2 = 10$		
	T_{rm}	$maxP1$	$maxP2$	T_{rm}	$maxP1$	$maxP2$	T_{rm}	$maxP1$	$maxP2$
$\lambda = 7$	0,7486	36	57	0,6658	42	36	0,5816	33	44
$\lambda = 8$	1,3324	41	119	1,0027	58	64	0,8297	43	54
$\lambda = 9$	113,159*	59*	2810*	1,9922	107	112	1,5087	59	194
$\lambda = 9,5$	–	–	–	3,8953	216	197	2,6813	81	205
$\lambda = 9,7$	–	–	–	6,3735	297	432	4,0685	94	308
$\lambda = 9,9$	–	–	–	20,8368	709	712	11,2027	123	813
$\lambda = 10$	–	–	–	213,88*	3401*	5111*	111,058*	115*	3914*

Tabela 4.1: Rezultatele obținute pentru modelul serie, repartiție exponențială ($DS = 10^6$)

În configurația asimetrică ($\mu_1 = 11, \mu_2 = 9$), stația 2 devine principalul punct critic de congestie al sistemului. Încă din regimul stabil, pe măsură ce λ crește de la 7 la 8, se observă cum valoarea $maxP_2$ se dublează (de la 57 la 119), în timp ce coada primei stații ($maxP_1$) înregistrează o creștere nesemnificativă (de la 36 la 41). Atingerea ratei de sosire $\lambda = 9$ corespunde exact pragului teoretic de stabilitate $\min(\mu_1, \mu_2) = 9$. În acest punct, sistemul își pierde starea staționară, fenomen evidențiat de creșterea explozivă și necontrolată a cozii la stația mai lentă ($maxP_2 = 2810^*$). Pentru $\lambda \geq 9,5$, instabilitatea devine totală, cozile nemaiputând fi evaluate în niciun fel de regim staționar.

În cazul simetric ($\mu_1 = \mu_2 = 10$), pragul de stabilitate impune $\lambda < 10$. Rularea simulării până în imediata vecinătate a acestui prag ($\lambda = 9,9$) ilustrează perfect comportamentul unui sistem aflat la granița saturației, dar încă în regim staționar: cozile ating valori ridicate, însă rămân echilibrate între cele două stații ($maxP_1 = 709, maxP_2 = 712$). Acest echilibru de încărcare diferă fundamental de configurația asimetrică. La atingerea pragului critic ($\lambda = 10$), sistemul părăsește regimul staționar, iar cozile explodează concomitent la ambele stații ($maxP_1 = 3401^*, maxP_2 = 5111^*$), dovedind că lipsa unei singure stații lente duce la o congestie a întregii rețele.

Configurația cu asimetrie redusă ($\mu_1 = 11, \mu_2 = 10$) reconfirmă tiparul observat în primul scenariu asimetric, chiar și în imediata vecinătate a pragului de saturație ($\lambda = 9,9$). Deși sistemul se află încă în regim staționar, congestia este deja preluată disproporționat de stația mai lentă ($maxP_2 = 813$), în timp

ce coada primei stații se menține la un nivel vizibil mai redus ($\max P_1 = 123$). Acest comportament demonstrează că orice diferență de capacitate, oricât de mică, canalizează tendința de blocaj exclusiv către resursa inferioară.

	$\mu_1 = 11, \mu_2 = 9$			$\mu_1 = 10, \mu_2 = 10$			$\mu_1 = 11, \mu_2 = 10$		
	T_{rm}	$\max P_1$	$\max P_2$	T_{rm}	$\max P_1$	$\max P_2$	T_{rm}	$\max P_1$	$\max P_2$
$\lambda = 7$	0,41	24	25	0,3518	22	9	0,3238	22	14
$\lambda = 8$	0,6786	27	49	0,4591	32	13	0,4148	26	23
$\lambda = 9$	519,311*	36*	7847*	0,7845	57	26	0,6824	37	61
$\lambda = 9,5$	–	–	–	1,4486	119	47	1,2139	51	102
$\lambda = 9,7$	–	–	–	2,3054	159	70	1,9189	54	176
$\lambda = 9,9$	–	–	–	7,0814	646	124	5,6004	61	480
$\lambda = 10$	–	–	–	169,747*	3547*	665*	200,696*	73*	6872*

Tabela 4.2: Rezultatele obținute pentru modelul serie, repartiție normală ($DS = 10^6$)

Rezultatele obținute pentru modelul în serie folosind distribuția normală (Tabelul 4.2) reconfirmă aceleași condiții teoretice de stabilitate observate în cazul distribuției exponențiale ($\lambda < 9$ pentru asimetria majoră și $\lambda < 10$ pentru celelalte configurații). Cu toate acestea, o comparație directă între cele două scenarii în regim staționar evidențiază o îmbunătățire vizibilă a performanțelor în cazul distribuției normale. De exemplu, pentru configurația $\mu_1 = 11, \mu_2 = 9$ la o rată de sosire $\lambda = 8$, timpul mediu de răspuns (T_{rm}) scade de la 1,3324 (în cazul exponențial) la doar 0,6786 (în cazul normal), reprezentând o reducere de aproape 50%. Un tipar similar se observă și la nivelul cozilor maxime, care sunt reduse semnificativ.

Această performanță superioară se explică strict prin reducerea dispersiei timpilor de servire. Spre deosebire de distribuția exponențială, care este caracterizată de o variabilitate ridicată și permite ocazional timpi de execuție extrem de lungi, distribuția normală utilizată constrânge fluctuațiile prin fixarea abaterii standard la o valoare redusă. Această predictibilitate elimină vârfurile haotice de congestie temporară. Prin urmare, chiar dacă serverul are exact aceeași viteză medie de procesare în ambele cazuri, ritmul constant și lipsa variațiilor extreme previn acumularea bruscă a jetoanelor, permițând sistemului să obțină timpi de răspuns considerabil mai buni.

Pentru a garanta validitatea datelor raportate și a elimina influența regimului tranzitoriu inerent pornirii sistemului dintr-o stare goală, este esențială confirmarea atingerii unui regim staționar. În acest sens, a fost efectuată o analiză de convergență a timpului mediu de răspuns (T_{rm}), variind durata simulării (DS) în trei etape distincte: 10^5 , $5 \cdot 10^5$ și 10^6 unități de timp. Tabelul 4.3 centralizează această evaluare pentru modelul în serie, testând comparativ ambele distribuții ale timpilor de servire pe cele trei configurații de bază. Pentru a obține o perspectivă completă asupra dinamicii rețelei, verificarea a fost realizată la două puncte de operare: un regim stabil, caracterizat printr-o rată de sosire moderată ($\lambda = 7$), și un regim critic, situat în imediata vecinătate a pragului teoretic de saturație specific fiecărei arhitecturi.

Înainte de a formula concluzii definitive privind performanța, este necesar un test care să ateste că simulatorul a rulat destul pentru a se stabili. Din moment ce sistemul pornește inițial fără nicio cerere, valorile măsurate prea devreme nu ar fi relevante pe termen lung. Astfel, Tabelul 4.3 monitorizează evoluția timpului mediu de răspuns (T_{rm}) la trei durate diferite de simulare (10^5 , $5 \cdot 10^5$ și 10^6). Pentru a avea certitudinea că sistemul atinge un regim staționar în orice condiții, am realizat această verificare atât pentru un flux normal de date ($\lambda = 7$), cât și pentru un trafic de mare intensitate, aflat la granița saturației.

Configurație (μ_1, μ_2)	Repartiție exponențială				Repartiție normală			
	λ	10^5	$5 \cdot 10^5$	10^6	λ	10^5	$5 \cdot 10^5$	10^6
(9, 10)	7	0,5123	0,4088	0,5106	7	0,4105	0,4121	0,4115
	8,9	5,1023	5,3537	5,3403	8,9	4,8237	4,6843	5,5339
(10, 10)	7	0,6631	0,6661	0,6660	7	0,3573	0,3503	0,3513
	9,9	20,5481	20,0338	21,3125	9,9	5,3561	6,5671	6,5913
(11, 10)	7	0,3213	0,3253	0,3251	7	0,3219	0,3237	0,3239
	9,9	6,6537	5,1478	5,6347	9,9	5,5232	5,4889	5,9524

Tabela 4.3: Verificarea regimului staționar pentru timpul mediu de răspuns T_{rm} (modelul în serie)

Validarea regimului staționar evidențiază două comportamente distincte ale rețelei. Pentru o stare de funcționare relaxată, sistemul atinge rapid echilibrul, performanțele evaluate rămânând constante odată ce faza tranzitorie este depășită. În contrast, apropierea de punctul de saturație face ca stabilizarea coziel să devină mult mai dificilă și fragilă. Astfel, se confirmă că menținerea regimului staționar sub stres extrem generează o variație normală a valorilor, aspect care impune rularea simulării pe intervale de timp mai lungi.

4.3. Modelul paralel

În analiza modelului paralel, o caracteristică importantă o reprezintă modul de procesare simultană a cererilor. Deoarece fiecare cerere ocupă ambele stații în același timp, sincronizarea realizându-se doar la ieșirea din sistem, fiecare ramură trebuie să proceseze independent întregul flux de intrare λ . Prin urmare, la fel ca în cazul topologiei în serie, viteza de procesare a întregului sistem nu reprezintă suma celor două, ci este limitată strict de stația cea mai lentă.

Astfel, condiția teoretică de stabilitate rămâne identică: rata de sosire trebuie să fie strict mai mică decât rata de servire a stației mai lente. Gradul de încărcare a sistemului, ρ , este dictat de această stație critică, fiind definit la fel ca în cazul modelului în serie.

Scenariile de testare păstrează aceeași structură ca în cazul modelului în serie, evaluând comportamentul sistemului de la regimul stabil ($\lambda = 7$) până în zona critică de saturație ($\rho \rightarrow 1$).

	$\mu_1 = 11, \mu_2 = 9$			$\mu_1 = 10, \mu_2 = 10$			$\mu_1 = 11, \mu_2 = 10$		
	T_{rm}	$maxP1$	$maxP2$	T_{rm}	$maxP1$	$maxP2$	T_{rm}	$maxP1$	$maxP2$
$\lambda = 7$	0,5594	34	60	0,4701	39	40	0,4169	35	38
$\lambda = 8$	1,0471	50	93	0,6994	58	63	0,5966	50	60
$\lambda = 9$	177,189*	61*	8281*	1,3905	124	147	1,109	71	108
$\lambda = 9,5$	–	–	–	2,781	187	211	2,1055	87	201
$\lambda = 9,7$	–	–	–	4,5155	280	267	3,3199	82	301
$\lambda = 9,9$	–	–	–	12,8357	635	682	9,9037	122	704
$\lambda = 10$	–	–	–	270,965*	5210*	7047*	168,411*	116*	4084*

Tabela 4.4: Rezultatele obținute pentru modelul paralel, repartiție exponențială ($DS = 10^6$)

Datele din tabelul de mai sus reproduc tiparul deja observat la modelul în serie: stația mai lentă preia disproporționat congestia, atât în cazul asimetriei accentuate, cât și în cel al asimetriei reduse, în timp ce la configurația simetrică ($\mu_1 = \mu_2 = 10$) diferența dintre $maxP_1$ și $maxP_2$ rămâne relativ mică.

	$\mu_1 = 11, \mu_2 = 9$			$\mu_1 = 10, \mu_2 = 10$			$\mu_1 = 11, \mu_2 = 10$		
	Trm	$maxP1$	$maxP2$	Trm	$maxP1$	$maxP2$	Trm	$maxP1$	$maxP2$
$\lambda = 7$	0,3158	21	33	0,2448	27	28	0,2279	19	23
$\lambda = 8$	0,5738	25	52	0,3414	32	35	0,3131	24	29
$\lambda = 9$	128,768*	36*	3142*	0,6302	57	60	0,5747	39	63
$\lambda = 9,5$	–	–	–	1,2091	120	114	1,0781	60	129
$\lambda = 9,7$	–	–	–	1,9399	172	170	1,7556	60	163
$\lambda = 9,9$	–	–	–	5,6133	332	347	5,3154	56	485
$\lambda = 10$	–	–	–	369,979*	6163*	6419*	214,026*	69*	5376*

 Tabela 4.5: Rezultatele obținute pentru modelul paralel, repartiție normală ($DS = 10^6$)

Ca și la modelul serie, limitarea dispersiei timpilor de servire nu modifică pragul teoretic de stabilitate, dar reduce sensibil timpul mediu de răspuns și dimensiunea cozilor.

Pentru a verifica dacă rezultatele raportate reflectă un regim staționar veritabil (și nu fluctuații specifice regimului tranzitoriu, cauzate de un timp de simulare insuficient), rulările au fost repetate la $\lambda = 7$ (regim stabil) și, în funcție de configurație, la $\lambda = 8,9$ sau $\lambda = 9,9$ (valori apropiate de pragul de stabilitate specific fiecărei configurații).

Configurație (μ_1, μ_2)	Repartiție exponențială				Repartiție normală			
	λ	10^5	$5 \cdot 10^5$	10^6	λ	10^5	$5 \cdot 10^5$	10^6
(11, 9)	7	0,5558	0,5588	0,5596	7	0,3156	0,3153	0,3158
	8,9	9,8148	11,3376	9,4924	8,9	5,2339	5,1956	5,3435
(10, 10)	7	0,4739	0,5701	0,5701	7	0,2440	0,2449	0,2448
	9,9	11,8738	13,1286	14,4294	9,9	5,2961	5,3749	5,4056
(11, 10)	7	0,4197	0,4157	0,4169	7	0,2284	0,2279	0,2279
	9,9	8,9367	9,7374	9,1871	9,9	6,3031	5,9953	5,6855

 Tabela 4.6: Verificarea regimului staționar pentru timpul mediu de răspuns T_{rm} (modelul paralel)

Regimul stabil ($\lambda = 7$) prezintă, de asemenea, valori practic constante ale T_{rm} pe întregul interval testat, confirmând convergența spre regim staționar.

4.4. Modelul sincron

Pentru a asigura o comparabilitate directă cu analiza modelelor anterioare, modelul sincron a fost testat folosind aceleași trei configurații ale ratelor de servire: $\mu_1 = 11, \mu_2 = 9$; $\mu_1 = 10, \mu_2 = 10$; $\mu_1 = 11, \mu_2 = 10$.

Deși la prima vedere s-ar putea considera că pragul de stabilitate este identic cu cel al modelului paralel ($\lambda < \min(\mu_1, \mu_2)$), mecanismul de funcționare impune o restricție suplimentară. Sistemul devine instabil la valori ale lui λ vizibil mai mici decât capacitatea stației mai lente. Această limitare provine din constrângerea de sincronizare: o cerere nouă nu poate intra în procesare decât după ce ambele ramuri ale cererii curente și-au încheiat execuția. Astfel, timpul efectiv de prelucrare al unei cereri nu este $1/\mu_1$ sau $1/\mu_2$, ci maximum dintre cele două: $S = \max(T_1, T_2)$, unde $T_1 \sim \text{Exp}(\mu_1)$ și $T_2 \sim \text{Exp}(\mu_2)$ sunt variabile aleatoare independente.

Media acestui maxim se calculează pornind de la identitatea $T_1 + T_2 = \max(T_1, T_2) + \min(T_1, T_2)$, valabilă pentru orice două variabile aleatoare, și de la faptul că minimum a două exponențiale independente este tot exponențial, cu rata sumă: $\min(T_1, T_2) \sim \text{Exp}(\mu_1 + \mu_2)$, o proprietate consacrată a

distribuției exponențiale în teoria cozilor de așteptare [10]. Se obține astfel:

$$E[S] = E[T_1] + E[T_2] - E[\min(T_1, T_2)] = \frac{1}{\mu_1} + \frac{1}{\mu_2} - \frac{1}{\mu_1 + \mu_2}.$$

Rata efectivă de servire a sistemului, $\mu_{ef} = 1/E[S]$, este, prin urmare, întotdeauna inferioară valorii $\min(\mu_1, \mu_2)$. Această scădere a performanței globale este cauzată de timpii de așteptare: ramura mai rapidă este blocată până la finalizarea procesării pe ramura mai lentă. Aplicând formula de mai sus, ratele efective pentru configurațiile testate sunt: $\mu_{ef} = 6,578$ (pentru $\mu_1 = 11$, $\mu_2 = 9$); $\mu_{ef} \approx 6,667$ (pentru $\mu_1 = \mu_2 = 10$); și $\mu_{ef} = 6,979$ (pentru $\mu_1 = 11$, $\mu_2 = 10$).

Din acest motiv, gradul de încărcare al sistemului sincron se raportează la această nouă rată efectivă ($\rho = \lambda/\mu_{ef}$). Pentru a capta corect comportamentul rețelei de la un regim stabil până la punctul de saturație ($\rho \rightarrow 1$), intervalul de variație pentru rata de sosire a fost ajustat. Valorile alese ($\lambda \in \{4; 5; 6; 6,3; 6,58; 6,67; 6,98\}$) oferă atât puncte de operare sigure, cât și limitele teoretice la care fiecare configurație intră în instabilitate.

	$\mu_1 = 11, \mu_2 = 9$		$\mu_1 = 10, \mu_2 = 10$		$\mu_1 = 11, \mu_2 = 10$	
	<i>Trm</i>	<i>maxP</i>	<i>Trm</i>	<i>maxP</i>	<i>Trm</i>	<i>maxP</i>
$\lambda = 4$	0,3354	22	0,3246	22	0,2933	22
$\lambda = 5$	0,5295	37	0,4987	38	0,4251	31
$\lambda = 6$	1,3785	82	1,1976	80	0,8226	55
$\lambda = 6,3$	2,8185	186	2,1398	154	1,1801	100
$\lambda = 6,58$	375,05*	6164*	8,7067	383	1,9591	139
$\lambda = 6,67$	–	–	640,438*	7749*	2,5006	152
$\lambda = 6,98$	–	–	–	–	422,407*	5891*

Tabela 4.7: Rezultatele obținute pentru modelul sincron, repartiție exponențială ($DS = 10^6$)

Rezultatele confirmă comportamentul analizat anterior: sistemul sincron devine instabil la valori ale lui λ apropiate de μ_{ef} , semnificativ mai devreme comparativ cu modelele în serie și paralel. De exemplu, pentru configurația $\mu_1 = 11$, $\mu_2 = 9$, rețeaua se menține într-un regim stabil până la $\lambda = 6,3$ ($T_{rm} = 2,8185$). La atingerea pragului teoretic $\lambda = 6,58$, performanța scade drastic, timpul mediu de răspuns ajungând la $T_{rm} = 375,05$. Această dinamică se regăsește și la celelalte configurații, unde instabilitatea se declanșează în funcție de valoarea μ_{ef} specifică fiecăreia. Spre deosebire de arhitecturile anterioare, timpul mediu de răspuns crește considerabil chiar și la rate de sosire aflate mult sub capacitatea individuală a stațiilor, fenomen cauzat exclusiv de mecanismul de sincronizare, care forțează sistemul să absoarbă fluctuațiile maxime de execuție de pe ambele ramuri.

Spre deosebire de distribuția exponențială, unde timpul mediu de sincronizare se deduce printr-un calcul direct, cazul variabilelor normale este mai complex. Diferența dintre cele două distribuții provine din dispersia timpilor de servire: la exponențială, apariția ocazională a unor timpi mult superiori mediei majorează semnificativ media maximului, în timp ce la normală, cu abatere standard redusă, asemenea valori extreme sunt practic absente. Întrucât nu există însă o ecuație de bază care să calculeze direct media maximului a două variabile normale independente, evaluarea a fost realizată folosind ecuațiile lui Clark [11], o soluție clasică din cercetarea operațională:

$$E[\max(T_1, T_2)] = m_1\Phi(a) + m_2\Phi(-a) + \theta\varphi(a), \quad \theta = \sqrt{\sigma_1^2 + \sigma_2^2}, \quad a = \frac{m_1 - m_2}{\theta},$$

unde Φ și φ reprezintă funcția de repartiție, respectiv funcția de densitate a distribuției normale standard, iar $m_i = 1/\mu_i$.

Aplicând această formulă, rezultă rate efective de servire (μ_{ef}) superioare cazului exponențial:

8,680 pentru $\mu_1 = 11, \mu_2 = 9$; 8,986 pentru $\mu_1 = \mu_2 = 10$; și 9,360 pentru $\mu_1 = 11, \mu_2 = 10$. Pe baza acestor noi limite, scenariile de simulare au utilizat rate de sosire $\lambda \in \{4; 5; 6; 7; 8; 8,5; 8,68; 8,99; 9,36\}$. La fel ca în analiza precedentă, intrarea în instabilitate este dictată strict de atingerea pragului μ_{ef} corespunzător fiecărei configurații. Diferența majoră constă în faptul că sistemul poate suporta un flux de intrare vizibil mai mare înainte de a ceda, confirmând astfel performanța superioară a modelului cu dispersie redusă.

	$\mu_1 = 11, \mu_2 = 9$		$\mu_1 = 10, \mu_2 = 10$		$\mu_1 = 11, \mu_2 = 10$	
	Trm	$maxP$	Trm	$maxP$	Trm	$maxP$
$\lambda = 4$	0,1656	10	0,1570	11	0,1475	11
$\lambda = 5$	0,1857	14	0,1827	15	0,1695	13
$\lambda = 6$	0,2477	21	0,2258	20	0,2045	19
$\lambda = 7$	0,3609	31	0,3123	34	0,2690	24
$\lambda = 8$	0,8092	72	0,5727	76	0,4279	45
$\lambda = 8,5$	2,8913	198	1,1139	127	0,6502	74
$\lambda = 8,68$	367,705*	5918*	1,7611	215	0,8066	76
$\lambda = 8,99$	—	—	486,062*	8120*	1,4309	119
$\lambda = 9,36$	—	—	—	—	557,524*	7625*

Tabela 4.8: Rezultatele obținute pentru modelul sincron, repartiție normală ($DS = 10^6$)

Pentru a confirma validitatea datelor, s-a efectuat analiza de convergență prin variația duratei de simulare (DS), urmărind metodologia stabilită anterior. Evaluarea a fost realizată comparativ pentru ambele distribuții, testând un punct de operare stabil ($\lambda = 4$) și unul situat în imediata vecinătate a limitei de saturație. Rezultatele centralizate în Tabelul 4.9 demonstrează stabilizarea valorilor la rate de sosire mici, confirmând atingerea unui regim staționar.

Configurație (μ_1, μ_2)	Repartiție exponențială				Repartiție normală			
	λ	10^5	$5 \cdot 10^5$	10^6	λ	10^5	$5 \cdot 10^5$	10^6
(11, 9)	4	0,3357	0,3357	0,3366	4	0,1657	0,1656	0,1657
	6,5	9,2281	11,7829	9,3349	8,6	5,6740	6,5116	6,4567
(10, 10)	4	0,3260	0,3247	0,3247	4	0,1566	0,1569	0,1569
	6,6	10,2423	12,1951	11,3978	8,9	5,9175	5,9959	6,1330
(11, 10)	4	0,2916	0,2933	0,2922	4	0,1477	0,1479	0,1476
	6,9	8,7756	9,3571	10,1705	9,3	10,7229	9,7798	9,0132

Tabela 4.9: Verificarea regimului staționar pentru timpul mediu de răspuns T_{rm} (modelul sincron)

Pentru regimul stabil ($\lambda = 4$), timpul mediu de răspuns (T_{rm}) a rămas practic constant și apropiat de valoarea minimă posibilă pe întregul interval de simulare testat (10^5 , $5 \cdot 10^5$ și 10^6), indiferent de configurație sau distribuție, confirmând un regim staționar veritabil, neafectat de fenomene tranzitorii. Tabelul 4.9 prezintă comparativ, pentru cele trei configurații și ambele distribuții, evoluția T_{rm} atât pentru punctul de operare stabil, cât și pentru rata de sosire aflată în imediata vecinătate a pragului de saturație, ilustrând direct comportamentul sistemului la creșterea duratei de simulare.

4.5. Sinteza modelelor de bază și impactul variabilității

Analiza detaliată a celor trei topologii fundamentale — în serie, paralelă și sincronă — prezentată în secțiunile anterioare, servește unui scop dublu: validarea riguroasă a motorului de simulare și

identificarea factorilor care influențează modelul compus.

În primul rând, rezultatele obținute au confirmat corectitudinea implementării simulatorului cu evenimente discrete. Faptul că indicatorii de performanță (timpul mediu de răspuns T_{rm} și dimensiunea cozilor) converg către valori stabile sub pragul teoretic de saturare validează atât mecanismele de gestionare a cozilor de tip FIFO, cât și acuratețea generatoarelor de evenimente stocastice pentru ambele tipuri de distribuții (exponențială și normală).

În al doilea rând, experimentele izolate au evidențiat fenomenele de propagare a congestiei care se regăsesc, la o scară mai mare, în evaluarea Rețelelor Petri Stohastice complexe. S-a demonstrat că, în cazul sub-rețelelor cu organizare în serie sau paralelă, capacitatea de procesare a întregului sistem este plafonată strict de stația cu cea mai mică rată de servire. Într-un model compus, efectul acestei limitări se amplifică de-a lungul lanțului: o singură stație lentă dictează ritmul întregii execuții, cauzând acumularea de jetoane (cereri) și sub-utilizarea resurselor.

Mai mult, testarea modelului sincron a evidențiat un cost ascuns al procesării paralele. Viteza reală a unei joncțiuni de sincronizare (μ_{ef}) este mereu mai mică decât viteza celei mai lente ramuri, pentru că sistemul trebuie să aștepte de fiecare dată procesul care termină ultimul. Într-o rețea complexă, aceste puncte de așteptare devin adevărate blocaje, scăzând considerabil capacitatea sistemului de a face față unui trafic intens. Un rol esențial în atenuarea acestor blocaje îl are predictibilitatea timpilor de execuție, aspect pe care îl vom detalia în continuare.

Pornind de la aceste constatări, premisa fundamentală în evaluarea topologiei complexe este că secțiunile rețelei cu o variație mare a timpului de procesare vor deveni principalele surse de blocaj, în timp ce secțiunile predictibile vor asigura menținerea unui flux global constant. Pentru a izola exact acest impact al variabilității asupra performanței, independent de capacitatea medie de procesare, am realizat o comparație directă între cele două distribuții analizate. Astfel, deși ambele scenarii păstrează aceeași medie a timpului de execuție ($1/\mu_i$), ele diferă fundamental prin dispersie: ridicată în cazul distribuției exponențiale și strict limitată ($\sigma_i = 1/(5\mu_i)$) în cazul celei normale.

Prima diferență notabilă se observă la nivelul pragului teoretic de stabilitate. Pentru modelele în serie și paralel, acest prag ($\lambda = \min(\mu_1, \mu_2)$) este insensibil la forma distribuției, fiind dictat exclusiv de valorile medii ale ratelor de servire. Rezultatele confirmă că ambele modele își pierd stabilitatea exact la aceleași valori critice, indiferent de distribuție (de exemplu, la $\lambda = 9$ pentru configurația asimetrică, sau la $\lambda = 10$ pentru celelalte cazuri). Prin urmare, la aceste topologii, variabilitatea influențează doar indicatorii de performanță în regim stabil, nu și punctul în care sistemul intră în colaps.

Situația se schimbă fundamental în cazul modelului sincron. Aici, rata efectivă de servire (μ_{ef}) este un rezultat direct al interacțiunii statistice dintre timpii individuali. Tabelul 4.10 ilustrează valorile obținute prin formula exactă (cazul exponențial), comparativ cu aproximația lui Clark (cazul normal).

(μ_1, μ_2)	μ_{ef} (exponențial)	μ_{ef} (normal)
(11, 9)	6,578	8,680
(10, 10)	6,667	8,986
(11, 10)	6,979	9,360

Tabela 4.10: Rata efectivă de servire (μ_{ef}) la sincronizare, în funcție de distribuție

Pentru a oferi o perspectivă integrată asupra dinamicii sistemului, Figura 4.1 ilustrează comparativ evoluția timpului mediu de răspuns (T_{rm}) în funcție de gradul de încărcare (ρ) pentru cele trei arhitecturi de bază, reprezentare care permite o comparație echitabilă, dincolo de diferența de prag de saturare evidențiată deja de μ_{ef} în Tabelul 4.10.

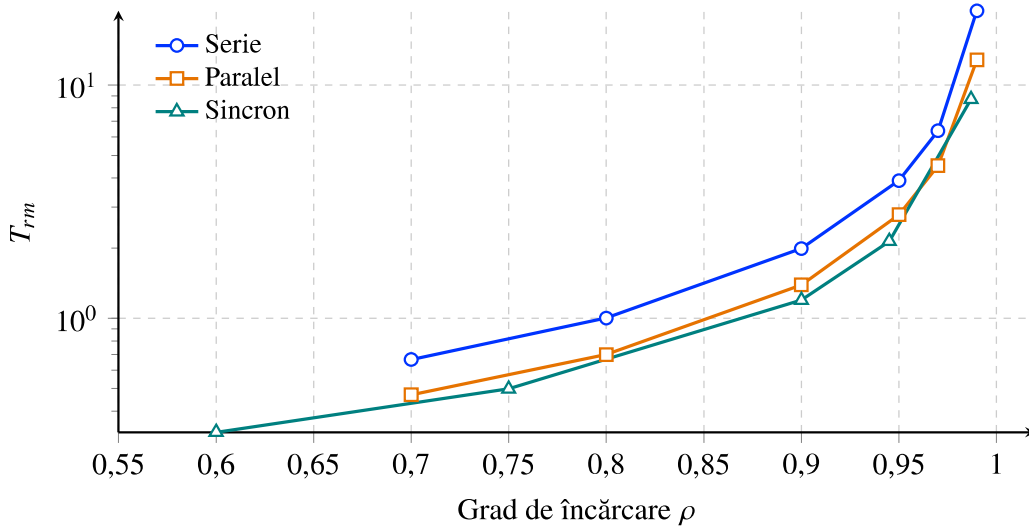


Figura 4.1: Timpul mediu de răspuns în funcție de gradul de încărcare pentru cele trei topologii

Se observă că, la același grad de încărcare, modelul serie înregistrează constant cele mai mari valori ale T_{rm} , în timp ce modelele paralel și sincron rămân apropiate.

Reducerea dispersiei crește semnificativ rata efectivă de servire a sistemului (cu aproximativ 30–35% pentru toate configurațiile testate). Aceasta demonstrează că, spre deosebire de fluxurile simple în serie sau paralel, la sincronizare o predictibilitate mai mare deplasează efectiv pragul de saturație, permițând preluarea unui trafic net superior înainte de instalarea instabilității.

Dincolo de pragul de saturație, impactul distribuției rămâne major și în regimul de funcționare stabil. Pentru modelul în serie (configurația asimetrică, la $\lambda = 8$), timpul mediu de răspuns scade de la 1,3324 (exponențial) la 0,6786 (normal), o reducere de aproape 50% la aceeași rată de sosire. Tiparul se repetă fidel și la modelul paralel, confirmând că vârfurile ocazionale de întârziere, specifice distribuției exponențiale, penalizează disproporționat sistemul, chiar dacă media vitezei de procesare este identică.

Evaluarea performanțelor modelului sincron necesită însă o abordare mai nuanțată. Deoarece pragul său de saturație se deplasează, o comparație la aceeași rată de sosire λ devine înșelătoare. De exemplu, la $\lambda = 6$, un timp de răspuns de peste cinci ori mai mic pentru distribuția normală reflectă, în principal, operarea la un grad de încărcare mult mai relaxat ($\rho \approx 0,69$) față de scenariul exponențial ($\rho \approx 0,91$).

Pentru a izola pur efectul variabilității, comparația trebuie realizată la un grad de încărcare echivalent (ρ). Analizând sistemul la $\rho \approx 0,91 - 0,92$, se observă că distribuția exponențială (la $\lambda = 6$) produce $T_{rm} = 1,3785$ și o coadă maximă de 82, în timp ce distribuția normală (la $\lambda = 8$) produce $T_{rm} = 0,8092$ și o coadă maximă de 72. Astfel, avantajul dispersiei reduse se menține clar și independent de deplasarea pragului de saturație, dovedind că un flux predictibil va genera mereu cozi mai mici și timpi de așteptare reduși.

În concluzie, variabilitatea timpilor de servire nu modifică *dacă* sistemele în serie și paralel devin instabile, ci doar *cât de sever* se manifestă degradarea performanței pe măsură ce $\rho \rightarrow 1$. La modelul sincron, în schimb, efectul este dublu: dispersia redusă crește rata efectivă de servire a sistemului (deplasând pragul de saturație) și, simultan, reduce timpul mediu de răspuns la un grad de încărcare comparabil. Această observație are o implicație practică directă: în sistemele care implică sincronizare între resurse paralele, reducerea variabilității timpului de servire poate produce un câștig de performanță comparabil cu (sau chiar superior) creșterii capacității medii de procesare.

4.6. Modelul compus

Odată validate principiile de bază, analiza se mută asupra modelului compus, care integrează servirea comună la intrare, politicile de prioritate nepreemptivă și ramificațiile cu sincronizare de tip fork-join. Scopul acestei secțiuni este de a demonstra că ansamblul funcționează corect, pornind de la testarea izolată a fluxurilor și ajungând până la competiția deplină pe resurse.

Validarea arhitecturii prin izolarea claselor de trafic

Pentru a confirma corectitudinea implementării mecanismelor din modelul compus, fiecare dintre cele patru clase de cereri a fost testată inițial izolat, menținând celelalte trei rate de sosire la valori neglijabile ($\approx 10^{-6}$). Rezultatele astfel obținute au fost comparate cu valorile deja stabilite în secțiunile 4.2 și 4.4, exploatând faptul că traseul unei clase izolate coincide structural cu topologiile de bază.

În cazul repartiției exponențiale, pentru clasele prioritare (1 și 2), traseul complet (stație comună urmată de stație cu prioritate) coincide, atunci când este izolat, cu topologia modelului în serie (configurația $\mu_1 = 11$, $\mu_2 = 10$), permițând o comparație directă cu rezultatele anterioare. Pentru clasele secundare (3 și 4), traseul se descompune în două etape independente: o coadă M/M/1 simplă (cu rata 11), urmată de etapa de sincronizare izolată la o singură culoare (caz simetric, $\mu_1 = \mu_2 = 10$). Justificarea teoretică a acestei descompunerii se bazează pe proprietatea lui Burke [12]: fluxul de plecări dintr-o coadă M/M/1 stabilă este el însuși Poisson, astfel încât etapa de sincronizare primește un flux de intrare cu o distribuție ideală.

λ	T_{rm} simulat	T_{rm} țintă	$\max_P$ simulat	$\max_P$ țintă	Sursa țintei
<i>Clasa 1</i> (λ_1 variabil, $\lambda_2 = \lambda_3 = \lambda_4 \approx 10^{-6}$)					
$\lambda_1 = 7$	0,5832	0,5816	$\max_{P_1} = 37$	33	Tabela 4.1, coloana (11, 10)
$\lambda_1 = 9$	1,5011	1,5087	$\max_{P_1} = 70$	59	Tabela 4.1, coloana (11, 10)
$\lambda_1 = 9, 9$	11,3087	11,2027	$\max_{P_1} = 115$	123	Tabela 4.1, coloana (11, 10)
<i>Clasa 2</i> (λ_2 variabil, $\lambda_1 = \lambda_3 = \lambda_4 \approx 10^{-6}$)					
$\lambda_2 = 7$	0,5824	0,5816	$\max_{P_1} = 31$	33	identic Clasa 1 (simetrie)
$\lambda_2 = 9$	1,4886	1,5087	$\max_{P_1} = 71$	59	identic Clasa 1
$\lambda_2 = 9, 9$	10,083	11,2027	$\max_{P_1} = 130$	123	identic Clasa 1
<i>Clasa 3</i> (λ_3 variabil, $\lambda_1 = \lambda_2 = \lambda_4 \approx 10^{-6}$)					
$\lambda_3 = 4$	0,4672	0,468	$\max_{P_6} = 20$	22	$\frac{1}{11-\lambda} + \text{Tabela 4.7, (10, 10)}$
$\lambda_3 = 6$	1,4085	1,398	$\max_{P_6} = 92$	80	$\frac{1}{11-\lambda} + \text{Tabela 4.7, (10, 10)}$
$\lambda_3 = 6, 3$	2,3411	2,353	$\max_{P_6} = 140$	154	$\frac{1}{11-\lambda} + \text{Tabela 4.7, (10, 10)}$
<i>Clasa 4</i> (λ_4 variabil, $\lambda_1 = \lambda_2 = \lambda_3 \approx 10^{-6}$)					
$\lambda_4 = 4$	0,4677	0,468	$\max_{P_6} = 20$	22	identic Clasa 3 (simetrie)
$\lambda_4 = 6$	1,3921	1,398	$\max_{P_6} = 94$	80	identic Clasa 3
$\lambda_4 = 6, 3$	2,3041	2,353	$\max_{P_6} = 123$	154	identic Clasa 3

Tabela 4.11: Validarea implementării modelului compus (repartiție exponențială)

Așa cum se observă în Tabelul 4.11, abaterile dintre valorile simulate și cele țintă sunt sub 2% pentru majoritatea punctelor testate. Singura excepție notabilă (clasa 2, la $\lambda = 9,9$, cu o abatere de $\approx 10\%$) este firească, fiind explicabilă prin variația naturală mare a sistemului în imediata vecinătate a pragului de saturatie. Aceste date confirmă că fiecare mecanism, atunci când este izolat, reproduce fidel comportamentul topologiei sale de bază.

Trecând la evaluarea sistemului sub repartiție normală, comparația pentru clasele 1 și 2 rămâne directă, prin același argument structural. Totuși, pentru clasele 3 și 4, compunerea analitică a celor două etape necesită o corecție fundamentală. Deoarece prima etapă nu mai este o coadă M/M/1, ci M/G/1 (sosiri Poisson, dar servire cu distribuție normală), timpul mediu de așteptare trebuie calculat prin formula Pollaczek–Khinchine [10]:

$$T = \frac{\lambda E[S^2]}{2(1 - \rho)} + E[S], \quad E[S^2] = \sigma^2 + \frac{1}{\mu^2},$$

unde $\rho = \lambda/\mu$. Astfel, valoarea țintă pentru fiecare λ testat devine suma dintre această contribuție și valoarea corespunzătoare etapei de sincronizare.

λ	T_{rm} simulat	T_{rm} țintă	$\max_P$ simulat	$\max_P$ țintă	Sursa țintei
<i>Clasa 1</i> (λ_1 variabil, $\lambda_2 = \lambda_3 = \lambda_4 \approx 10^{-6}$)					
$\lambda_1 = 7$	0,3239	0,3238	$\max_{P_1} = 17$	22	Tabela 4.2, coloana (11, 10)
$\lambda_1 = 9$	0,6785	0,6824	$\max_{P_1} = 39$	37	Tabela 4.2, coloana (11, 10)
$\lambda_1 = 9, 9$	5,3729	5,6004	$\max_{P_1} = 61$	61	Tabela 4.2, coloana (11, 10)
<i>Clasa 2</i> (λ_2 variabil, $\lambda_1 = \lambda_3 = \lambda_4 \approx 10^{-6}$)					
$\lambda_2 = 7$	0,3238	0,3238	$\max_{P_1} = 16$	22	identic Clasa 1
$\lambda_2 = 9$	0,6801	0,6824	$\max_{P_1} = 40$	37	identic Clasa 1
$\lambda_2 = 9, 9$	5,4344	5,6004	$\max_{P_1} = 57$	61	identic Clasa 1
<i>Clasa 3</i> (λ_3 variabil, $\lambda_1 = \lambda_2 = \lambda_4 \approx 10^{-6}$)					
$\lambda_3 = 4$	0,2503	0,2749	$\max_{P_6} = 7$	11	P-K (etapa 1) + Tabela 4.8, (10, 10)
$\lambda_3 = 6$	0,3209	0,3734	$\max_{P_6} = 15$	20	P-K (etapa 1) + Tabela 4.8, (10, 10)
$\lambda_3 = 8, 5$	1,2055	1,3656	$\max_{P_6} = 93$	127	P-K (etapa 1) + Tabela 4.8, (10, 10)
<i>Clasa 4</i> (λ_4 variabil, $\lambda_1 = \lambda_2 = \lambda_3 \approx 10^{-6}$)					
$\lambda_4 = 4$	0,2504	0,2749	$\max_{P_6} = 7$	11	identic Clasa 3
$\lambda_4 = 6$	0,3280	0,3734	$\max_{P_6} = 15$	20	identic Clasa 3
$\lambda_4 = 8, 5$	1,2048	1,3656	$\max_{P_6} = 96$	127	identic Clasa 3

Tabela 4.12: Validarea implementării modelului compus (repartiție normală)

Spre deosebire de cazul exponențial, datele din Tabelul ?? arată, pentru clasele 3 și 4, o abatere între 9% și 14%, cu T_{rm} simulat mereu mai mic decât cel analitic, în timp ce pentru clasele 1 și 2 abaterea rămâne neglijabilă, confirmând validitatea comparației directe cu topologia serie. Această deviație nu reprezintă o eroare a simulatorului, ci reliefează o limitare inerentă a modelelor matematice clasice: proprietatea lui Burke [12] se aplică exclusiv cozilor cu servire exponențială. La distribuția normală, fluxul de ieșire din prima etapă este mult mai regulat decât un flux Poisson teoretic, astfel încât etapa de sincronizare primește, în realitate, un trafic mult mai puțin haotic. Acest flux stabilizat produce un timp de așteptare și o coadă maximă inferioare previziunilor analitice, demonstrând capacitatea simulatorului de a capta nuanțe de regularizare a traficului pe care calculul teoretic le ratează.

Dinamica sistemului sub trafic mixt și concurența pe resurse

Deși validarea izolată a claselor confirmă corectitudinea structurală a modelului, un sistem informatic real funcționează rareori în astfel de condiții. Pentru a înțelege pe deplin dinamica rețelei, analiza trebuie să evalueze interacțiunea directă dintre clase, urmărind propagarea congestiei prin politica de priorități și concurența simultană pe resursele partajate.

Primul pas în această direcție l-a reprezentat interacțiunea dintre o clasă prioritară și una secundară. Pentru a cuantifica impactul traficului inferior asupra celui superior, s-a fixat $\lambda_1 = 5$ (regim stabil, $\rho_1 = 0,5$) și s-a variat $\lambda_3 \in \{10^{-6}, 3, 4, 5\}$, păstrând restul claselor inactice ($\lambda_2 = \lambda_4 = 10^{-6}$). Simulările au fost rulate pentru o durată $DS = 10^6$. O predicție analitică pentru clasa 1 a fost construită folosind formula lui Cobham [13] pentru cozi M/M/1 cu prioritate nepreemptivă:

$$R = \frac{\lambda_1}{\mu_{21}^2} + \frac{\lambda_3}{\mu_{23}^2}, \quad W_{q,1} = \frac{R}{1 - \rho_1}, \quad \rho_1 = \frac{\lambda_1}{\mu_{21}}$$

la care s-a adăugat timpul petrecut în prima etapă și timpul propriu de servire la stația finală.

λ_3	Clasa 1 ($\lambda_1 = 5$ fix, prioritară)		Clasa 3 (λ_3 variabil, secundară)	
	T_{rm} simulat	CT	T_{rm} simulat	CT
10^{-6}	0,3664	5 001 455	0,3717	2
3	0,4158	5 001 478	1,0667	3 004 571
4	0,4327	5 006 499	1,7967	3 999 394
5	0,4495	5 001 046	5,0755	5 002 352

Tabela 4.13: Efectul creșterii traficului secundar (λ_3) asupra performanței claselor 1 și 3

Comparația cu valorile simulate (de exemplu, 0,4163 teoretic vs. 0,4158 simulat la $\lambda_3 = 3$) indică abateri de sub 0,15%, oferind o confirmare solidă a mecanismului de prioritizare. Se observă că, deși clasa prioritară este eficient izolată, influența traficului secundar nu este complet eliminată, timpul său de răspuns crescând totuși cu aproximativ 23% pe parcursul testului. Acest fenomen decurge din natura nepreemptivă a sistemului: o cerere prioritară nu poate întrerupe o cerere secundară aflată deja în execuție. În schimb, timpul de răspuns al clasei secundare explodează (crește de peste 13 ori), ea suportând atât efectul propriei congestii, cât și penalizarea generată de clasa superioară.

Extinzând analiza către scenariul cel mai apropiat de utilizarea reală, sistemul a fost testat cu toate cele patru clase active simultan. Această configurație aduce în scenă competiția directă dintre fluxurile inferioare (clasele 3 și 4) pentru resursa unică de sincronizare. Setarea parametrilor ($\lambda_1 = \lambda_2 = 2$, $\lambda_3 = \lambda_4 = 3$) a fost aleasă pentru a aduce etapa de sincronizare la un grad de încărcare de aproximativ 90%, o zonă de stres ridicat, dar teoretic stabilă. Simularea a rulat pentru o durată $DS = 10^6$, înregistrându-se pe parcurs cozi maxime de 13 cereri pentru locația P_1 și 80 de cereri pentru locația de sincronizare P_6 .

Clasa	T_{rm}	CT/DS (debit)
1	0,2985	2,0007
2	0,2984	2,0022
3	1,4137	3,0035
4	1,4159	2,9995

Tabela 4.14: Performanța claselor în regim de trafic complet

Valorile centralizate în Tabelul 4.14 evidențiază o funcționare robustă și confirmă deplin simetria structurală a modelului. Timpii de răspuns pentru clasele pereche sunt practic identici (diferențe sub 0,1%), iar rata de ieșire se aliniază riguros cu ratele de sosire impuse, demonstrând că sistemul nu pierde jetoane și nu acumulează cozi infinite nici sub stres sever. Diferența majoră de performanță reprezintă consecința directă a efectelor combinate ale absenței priorității și ale costului de sincronizare, mecanisme care interacționează predictibil, fără a genera fenomene haotice.

Rezultatele experimentale prezentate de-a lungul acestui capitol validează implementarea sistemului atât calitativ, cât și cantitativ. S-a demonstrat că variația timpilor de servire dictează magnitudinea întârzierilor în rețea, distribuțiile cu dispersie redusă dovedindu-se superioare, inclusiv la îmbunătățirea pragului de saturație în cazul joncțiunilor de sincronizare. Simulatorul a demonstrat capacitatea de a capta fenomene de regularizare a traficului care scapă ecuațiilor clasice, iar modelul stocastic compus a reușit să integreze cu succes politici complexe de rutare, prioritizare și servire comună. Chiar și sub condiții de trafic intens și competiție ridicată pe resurse, sistemul și-a demonstrat stabilitatea, având un comportament predictibil și coerent până în imediata vecinătate a limitelor teoretice de saturație.

Capitolul 5. Concluzii

Lucrarea de față și-a propus dezvoltarea unui simulator bazat pe Rețele Petri Stohastice, capabil să modeleze, testeze și compare mai multe topologii de sisteme de servire. Toate obiectivele specifice enumerate în Secțiunea 1.2 au fost atinse integral, fără să fie necesară restrângerea sau modificarea vreunui dintre ele pe parcursul dezvoltării: au fost construite patru modele funcționale, trei topologii fundamentale (serie, paralel, sincron) și un model compus care le integrează mecanismele, implementate integral în C++, cu timpi de sosire și de servire eșantionați atât din distribuții exponențiale, cât și normale. Comportamentul fiecărei topologii a fost analizat sistematic prin variația ratei de sosire, a ratelor de servire și a duratei de simulare, urmărindu-se timpul mediu de răspuns, dimensiunea maximă a cozilor de așteptare și identificarea pragurilor de instabilitate.

Contribuția originală a lucrării constă în proiectarea și implementarea modelului compus, care extinde cele trei topologii fundamentale printr-o arhitectură ce integrează simultan patru clase de cereri cu trasee distincte, o stație comună de servire cu rată dependentă de clasă, o regulă de prioritate nepreemptivă între clasele concurente și o etapă finală de sincronizare de tip fork-join. Spre deosebire de cele trei modele fundamentale, pentru care condiția de stabilitate a putut fi derivată analitic ($\rho = \lambda / \min(\mu_1, \mu_2)$ pentru serie și paralel, respectiv $\rho = \lambda / \mu_{ef}$ pentru sincron, cu μ_{ef} calculat exact pentru distribuția exponențială și aproximat prin ecuațiile lui Clark pentru cea normală), complexitatea combinată a priorității și sincronizării face improbabilă o formulă analitică echivalentă pentru modelul compus, acesta fiind motivul principal pentru care simularea cu evenimente discrete, și nu analiza matematică directă, a reprezentat alegerea justificată pentru acest studiu.

Corectitudinea implementării modelului compus a fost confirmată printr-o validare pe bucăți, izolând fiecare dintre cele patru clase de cereri și comparând rezultatele cu topologiile de bază deja testate (Secțiunea 4.5). Pe lângă confirmarea directă a mecanismelor de servire comună, prioritate și sincronizare, această validare a scos la iveală o observație structurală suplimentară: la distribuția normală, unde proprietatea lui Burke nu mai garantează un flux Poisson între etape, sistemul manifestă o performanță ușor superioară celei prezise prin evaluarea independentă a sub-modelelor, acest fenomen reprezentând un efect secundar favorabil al reducerii dispersiei timpilor de servire, distinct de reducerea directă a lui T_{rm} deja documentată în Secțiunea 4.5.

Rezultatele experimentale au confirmat câteva concluzii cu valoare practică directă. Pentru modelele serie și paralel, pragul teoretic de stabilitate depinde exclusiv de ratele medii de servire, fiind identic indiferent de distribuția aleasă (Tabelele 4.1–4.5); variabilitatea timpului de servire influențează însă puternic valorile indicatorilor în regim stabil, distribuția normală reducând timpul mediu de răspuns cu până la 50% față de cea exponențială, la aceeași rată de sosire. Modelul sincron s-a comportat calitativ diferit: rata efectivă de servire (μ_{ef}) rezultată din mecanismul de așteptare reciprocă a fost întotdeauna inferioară capacității individuale a stațiilor mai lente (Tabelul 4.7), iar reducerea dispersiei timpilor de servire a deplasat efectiv acest prag cu 30–35%, confirmând că, în sistemele cu sincronizare, controlul variabilității poate produce un câștig de performanță comparabil cu mărirea capacității medii de procesare. Analiza de convergență (Tabelele 4.3, 4.6, 4.9) a validat metodologia de testare, confirmând că duratele mari de simulare alese elimină efectele regimului tranzitoriu inițial.

Comparativ cu abordările analitice discutate în Capitolul 2, alegerea rețelelor Petri stohastice s-a dovedit justificată: spre deosebire de un lanț Markov construit direct (supus exploziei spațiului de stări odată cu adăugarea claselor și a mecanismelor de prioritate și sincronizare) sau de formulele clasice ale teoriei cozilor de așteptare (aplicabile riguros doar unor topologii cu o singură stație), formalismul ales a permis atât o reprezentare grafică explicită a structurii sistemului, utilă pentru identificarea intuitivă a resurselor critice, cât și o evaluare cantitativă riguroasă a performanței, inclusiv pentru configurația compusă, mult mai greu de tratat prin metodele clasice enumerate. Deși evaluarea s-a bazat pe o singură rulare de lungă durată, susținută de analiza de convergență realizată, și nu pe

replicări independente însoțite de intervale de încredere, rezultatul rămâne relevant prin faptul că arată cum, prin compunerea unor mecanisme verificate individual, poate fi explicat comportamentul unei configurații mai complexe decât oricare dintre topologiile de bază, un tip de raționament greu de reprodus la aceeași scară prin metodele analitice clasice discutate anterior.

5.1. Direcții viitoare de dezvoltare

Mai multe direcții pot extinde rezultatele acestei lucrări. O primă direcție vizează derivarea unei condiții aproximative de stabilitate pentru modelul compus, care să combine efectele priorității nepreemptive cu cele ale sincronizării fork-join, chiar dacă doar sub formă de margini superioare/inferioare, nu ca formulă exactă; o asemenea condiție ar oferi o garanție teoretică asupra comportamentului sistemului, dincolo de observațiile empirice obținute prin simulare în configurațiile testate. O a doua direcție constă în rularea unor replicări independente multiple pentru fiecare configurație testată, cu scopul de a raporta intervale de încredere pentru indicatorii de performanță, în locul estimărilor punctuale utilizate în cadrul acestei cercetări, eliminând astfel limitarea metodologică asumată în secțiunea anterioară. Modelul poate fi extins la un număr mai mare de clase de cereri sau la reguli de prioritate preemptivă, precum și generalizat de la sincronizarea a două ramuri la sincronizarea a n resurse concurente, pentru a verifica dacă abordarea propusă își păstrează valabilitatea și dincolo de configurația particulară analizată în această lucrare. Nu în ultimul rând, dezvoltarea unei interfețe grafice care să vizualizeze în timp real evoluția marcajului rețelei ar facilita atât depanarea, cât și înțelegerea intuitivă a comportamentului sistemului simulat, utilă îndeosebi pe măsură ce modelul ar crește în complexitate odată cu extinderile menționate anterior.

5.2. Lecții învățate pe parcursul dezvoltării proiectului de diplomă

Dezvoltarea acestei lucrări a confirmat, practic, un principiu pe care literatura de specialitate îl menționează adesea: complexitatea unui sistem concurent crește mult mai rapid decât suma mecanismelor sale individuale. Fiecare dintre cele trei topologii fundamentale a fost, luată separat, relativ ușor de validat, atât structural, cât și prin compararea rezultatelor simulării cu pragurile de stabilitate derivate analitic. Integrarea lor într-un singur model compus a ridicat însă probleme pe care testarea izolată a componentelor nu le putea anticipa, în special în privința traseului exact pe care o cerere îl urmează prin sistem. Această experiență a întărit convingerea că, într-un proiect de simulare, corectitudinea modelului formal și corectitudinea codului care îl execută trebuie validate împreună, nu succesiv.

Totodată, existența acestor module de bază, validate individual, a redus semnificativ suprafața de căutare a erorilor în etapa de integrare: odată confirmată corectitudinea fiecărei topologii izolate, problemele apărute la compunere au putut fi atribuite cu încredere logicii de rutare dintre module, nu mecanismelor interne ale acestora. Prin urmare, o colecție de modele primare pre-validate poate constitui o bază solidă pentru dezvoltarea unei game variate de configurații superioare, deși integrarea lor rămâne, prin natura sa, o etapă distinctă ce necesită propria procedură de validare.

Nu în ultimul rând, observarea directă, prin simulare, a unor fenomene (precum regularizarea traficului la joncțiunile de sincronizare) care nu rezultă din formulele analitice clasice a confirmat, dincolo de argumentul teoretic din Capitolul 2, valoarea practică a simulării ca instrument de explorare a comportamentului sistemelor complexe, nu doar de validare a unor rezultate deja anticipate analitic.

Bibliografie

- [1] N. V. Nenkov and E. Z. Spasova, "Implementation of a Neural Network Using Simulator and Petri Nets," *International Journal of Advanced Computer Science and Applications*, Vol. 7, No. 1, 2016, DOI: 10.14569/IJACSA.2016.070155.
- [2] J. L. Peterson, *Petri Net Theory and the Modeling of Systems*, Englewood Cliffs, NJ: Prentice-Hall, 1981.
- [3] M. Ajmone Marsan, "Stochastic Petri Nets: An Elementary Introduction," in *Advances in Petri Nets 1989*, *Lecture Notes in Computer Science*, vol. 424, Springer, 1990, pp. 1–29, DOI: 10.1007/3-540-52494-0_23.
- [4] Kendall, D.G., *Stochastic Processes Occurring in the Theory of Queues and their Analysis by the Method of the Imbedded Markov Chain*, *The Annals of Mathematical Statistics*, vol. 24, nr. 3, pp. 338–354, 1953.
- [5] Flatto, L., Hahn, S., *Two parallel queues created by arrivals with two demands I*, *SIAM Journal on Applied Mathematics*, vol. 44, no. 5, pp. 1041–1053, 1984.
- [6] Nelson, R., Tantawi, A.N., *Approximate analysis of fork/join synchronization in parallel queues*, *IEEE Transactions on Computers*, vol. 37, no. 6, pp. 739–743, 1988.
- [7] Harchol-Balter, M., Osogami, T., Scheller-Wolf, A., Wierman, A., *Multi-server queueing systems with multiple priority classes*, *Queueing Systems: Theory and Applications*, vol. 51, pp. 331–360, 2005.
- [8] T. Murata, "Petri Nets: Properties, Analysis and Applications," *Proceedings of the IEEE*, vol. 77, no. 4, pp. 541–580, 1989, DOI: 10.1109/5.24143.
- [9] Cașcaval, P., *Modelarea și simularea sistemelor cu evenimente discrete*, Editura Performantica, Iași, 2006.
- [10] Trivedi, K.S., *Probability and Statistics with Reliability, Queueing, and Computer Science Applications*, John Wiley & Sons, New York, 2002.
- [11] C. E. Clark, "The Greatest of a Finite Set of Random Variables," *Operations Research*, vol. 9, no. 2, pp. 145–162, 1961, DOI: 10.1287/opre.9.2.145.
- [12] P. J. Burke, "The Output of a Queueing System," *Operations Research*, vol. 4, no. 6, pp. 699–704, 1956, DOI: 10.1287/opre.4.6.699.
- [13] A. Cobham, "Priority Assignment in Waiting Line Problems," *Operations Research*, vol. 2, no. 1, pp. 70–76, 1954, DOI: 10.1287/opre.2.1.70.

Anexe

Această secțiune conține codul sursă, în limbajul C++, pentru mecanismele esențiale ale celor patru modele de rețea Petri stohastică dezvoltate și discutate în Capitolul 3: modelul cu organizare în serie, modelul paralel, modelul sincron și modelul compus. Structurile de date globale, bucla principală de simulare și mecanismul de avansare a ceasului urmează îndeaproape algoritmi descriși în Secțiunile 3.3 și 3.5 și nu sunt reproduse aici; sunt prezentate doar componentele specifice fiecărui model.

Anexa 1. Generatoare de variabile aleatoare

Ambele funcții de mai jos sunt comune tuturor celor patru modele prezentate.

Generatorul exponențial

```
1 double genExp(double lambda) {
2     double u, x;
3     u = (double)rand() / ((double)RAND_MAX + 1.0);
4     x = -1.0 / lambda * log(1.0 - u);
5     return x;
6 }
```

Generatorul normal

```
1 double genGauss(double medie, double sigma) {
2     double s = 0;
3     int i;
4     for (i = 1; i <= 12; i++)
5         s += (double)rand() / RAND_MAX;
6     return medie + sigma * (s - 6);
7 }
```

Anexa 2. Modelul serie

Structura generală a buclei de simulare urmează Algoritmul 1 din Secțiunea 3.3, cu `nr_t = 3`. Funcția `executaTranzitie` de mai jos implementează logica specifică celor trei tranziții temporizate ale modelului: sosirea unei cereri în prima stație, trecerea la a doua stație și ieșirea din sistem.

```
1 void executaTranzitie(int te) {
2     switch (te) {
3     case 1: //sosire client in statia 1
4         M[1]++;
5         if (M[1] > maxp1) maxp1 = M[1];
6         D[1] = genExp(lambda);
7         if (M[1] == 1)
8             D[2] = genExp(miul);
9         break;
10
11     case 2: //trecerea de la statia 1 la statia 2
12         M[1]--;
```

```

13     M[2]++;
14     if (M[2] > maxp2) maxp2 = M[2];
15     if (M[1] > 0)
16         D[2] = genExp(miu1);
17     else
18         D[2] = -1;
19     if (M[2] == 1)
20         D[3] = genExp(miu2);
21     break;
22
23 case 3:    //iesirea din sistem
24     M[2]--;
25     CT++;
26     if (M[2] > 0)
27         D[3] = genExp(miu2);
28     else
29         D[3] = -1;
30     break;
31 }
32 }

```

Anexa 3. Modelul paralel

Reuniunea fork-join

Spre deosebire de celelalte mecanisme prezentate în această anexă, reuniunea fork-join a modelului paralel nu este implementată ca funcție separată, ci direct în cadrul buclei principale de simulare (main). Fragmentul de mai jos execută tranziția imediată t4 ori de câte ori ambele rezultate parțiale sunt disponibile.

```

1 while (M[3] > 0 && M[4] > 0) {
2     M[3]--;
3     M[4]--;
4     CT++;
5
6     if (nQ > 0) {
7         STr += ceas - Q[1];
8         for (int i = 1; i < nQ; i++) Q[i] = Q[i + 1];
9         nQ--;
10    }
11 }

```

Execuția tranzițiilor temporizate

Funcția `executaTranzitie` gestionează sosirea perechii de cereri și finalizarea prelucrării pe fiecare dintre cele două procesoare independente.

```

1 void executaTranzitie(int te) {
2     switch (te) {
3     case 1:
4         M[1]++;
5         M[2]++;
6         if (M[1] > maxp1) maxp1 = M[1];
7         if (M[2] > maxp2) maxp2 = M[2];
8         nQ++;
9         Q[nQ] = ceas;
10        D[1] = genExp(lambda);

```

```

11     if (M[1] >= 1 && D[2] < 0)
12         D[2] = genExp(miu1);
13     if (M[2] >= 1 && D[3] < 0)
14         D[3] = genExp(miu2);
15     break;
16
17 case 2:
18     M[1]--;
19     M[3]++;
20     if (M[1] > 0)
21         D[2] = genExp(miu1);
22     else
23         D[2] = -1;
24     break;
25
26 case 3:
27     M[2]--;
28     M[4]++;
29     if (M[2] > 0)
30         D[3] = genExp(miu2);
31     else
32         D[3] = -1;
33     break;
34 }
35 }

```

Anexa 4. Modelul sincron

Faza de prioritate a tranzițiilor netemporizate

La fel ca la modelul paralel, faza de prioritate a tranzițiilor netemporizate (finalizarea sincronizată t_5 și pornirea unei noi prelucrări t_2) este implementată direct în bucla principală de simulare, nu ca funcție separată.

```

1  bool net = true;
2  while (net) {
3      net = false;
4
5      if (M[5] > 0 && M[6] > 0) { // t5
6          M[5]--;
7          M[6]--;
8          M[2]++;
9
10         CT++;
11
12         if (nQ > 0) {
13             Str += ceas - Q[1];
14             for (int i = 1; i < nQ; i++) Q[i] = Q[i + 1];
15             nQ--;
16         }
17
18         net = true;
19     }
20
21     if (M[1] > 0 && M[2] > 0) { // t2
22         M[1]--;
23         M[2]--;
24         M[3]++;
25         M[4]++;

```

```

26
27     if (D[2] < 0)
28         D[2] = genExp(miu1); // t3
29         //D[2] = genGauss(1./miu1, sigma1);
30     if (D[3] < 0)
31         D[3] = genExp(miu2); // t4
32         //D[3] = genGauss(1./miu2, sigma2);
33
34     net = true;
35 }
36 }

```

Execuția tranzițiilor temporizate

Funcția `executaTranzitie` gestionează sosirea cererii și finalizarea prelucrării pe fiecare dintre cele două procesoare.

```

1 void executaTranzitie(int te) {
2     switch (te) {
3     case 1: // t1 - SOSIRE CERERE
4         M[1]++;
5         if (M[1] > maxp1) maxp1 = M[1];
6         nQ++;
7         Q[nQ] = ceas;
8         D[1] = genExp(lambda);
9         break;
10
11    case 2: // t3 - Procesorul 1 si-a incheiat prelucrarea
12        M[3]--;
13        M[5]++;
14        D[2] = -1;
15        break;
16
17    case 3: // t4 - Procesorul 2 si-a incheiat prelucrarea
18        M[4]--;
19        M[6]++;
20        D[3] = -1;
21        break;
22    }
23 }

```

Anexa 5. Modelul compus

Rezolvarea tranzițiilor imediate

Funcția `ruleazaTranzitie`, apelată la fiecare iterație a buclei principale, corespunde Algoritmului 2 din Secțiunea 3.3 și implementează regulile de selecție pentru resursele partajate: servirea comună FIFO (clasele 1 și 2), prioritatea nepreemptivă la cele două stații secundare și declanșarea/finalizarea etapei de sincronizare fork-join.

```

1 void ruleazaTranzitie() {
2     int i;
3     bool ok = true;
4     while (ok) {
5         ok = false;
6
7         //t5 devine validata

```

```

8   if (D[5] < 0 && nP1 > 0) {
9       t5_c = P1_c[1]; t5_s = P1_s[1];
10      for (i = 1; i < nP1; i++) { P1_s[i] = P1_s[i + 1]; P1_c[i] = P1_c[i + 1]; }
11      nP1--;
12      D[5] = (t5_c == 1) ? genExp(miu11) : genExp(miu12);
13      ok = true;
14  }
15
16  //t6 devine validata
17  if (D[6] < 0 && (nP2 > 0 || nP3 > 0)) {
18      if (nP2 > 0) {
19          t6_c = 1; t6_s = P2[1];
20          for (i = 1; i < nP2; i++) P2[i] = P2[i + 1];
21          nP2--;
22      }
23      else {
24          t6_c = 3; t6_s = P3[1];
25          for (i = 1; i < nP3; i++) P3[i] = P3[i + 1];
26          nP3--;
27      }
28      D[6] = (t6_c == 1) ? genExp(miu21) : genExp(miu23);
29      ok = true;
30  }
31
32  //t7 devine validata
33  if (D[7] < 0 && (nP5 > 0 || nP4 > 0)) {
34      if (nP5 > 0) {
35          t7_c = 2; t7_s = P5[1];
36          for (i = 1; i < nP5; i++) P5[i] = P5[i + 1];
37          nP5--;
38      }
39      else {
40          t7_c = 4; t7_s = P4[1];
41          for (i = 1; i < nP4; i++) P4[i] = P4[i + 1];
42          nP4--;
43      }
44      D[7] = (t7_c == 2) ? genExp(miu32) : genExp(miu34);
45      ok = true;
46  }
47
48  //t8
49  if (!P7P8 && P11 && nP6 > 0) {
50      P7P8_c = P6_c[1]; P7P8_s = P6_s[1];
51      for (i = 1; i < nP6; i++) { P6_s[i] = P6_s[i + 1]; P6_c[i] = P6_c[i + 1]; }
52      nP6--;
53      P7P8 = 1; P11 = 0; t9 = 0; t10 = 0;
54      D[9] = (P7P8_c == 3) ? genExp(miu43) : genExp(miu44);
55      D[10] = (P7P8_c == 3) ? genExp(miu43) : genExp(miu44);
56      ok = true;
57  }
58
59  //t11-sincronizarea
60  if (P7P8 && t9 && t10) {
61      STr[P7P8_c] += ceas - P7P8_s;
62      CT[P7P8_c]++;
63      P7P8 = 0; P11 = 1;
64      ok = true;
65  }
66  }
67  }

```

Execuția tranzițiilor temporizate

Funcția `executaTranzitie` tratează sosirea cererilor din toate cele patru clase și finalizarea fiecărei etape de prelucrare, direcționând fluxul conform arhitecturii descrise în Secțiunea 3.2 (Figura 3.5).

```

1 void executaTranzitie(int te) {
2     switch (te) {
3         case 1: //sosire cerere culoare 1
4             nP1++; P1_s[nP1] = ceas; P1_c[nP1] = 1;
5             if (nP1 > maxP1) maxP1 = nP1;
6             D[1] = genExp(lmd1);
7             break;
8         case 2: //sosire cerere culoare 2
9             nP1++; P1_s[nP1] = ceas; P1_c[nP1] = 2;
10            if (nP1 > maxP1) maxP1 = nP1;
11            D[2] = genExp(lmd2);
12            break;
13        case 3: //sosire culoare 3
14            nP3++; P3[nP3] = ceas;
15            if (nP3 > maxP3) maxP3 = nP3;
16            D[3] = genExp(lmd3);
17            break;
18        case 4: //sosire culoare 4
19            nP4++; P4[nP4] = ceas;
20            if (nP4 > maxP4) maxP4 = nP4;
21            D[4] = genExp(lmd4);
22            break;
23        case 5: //finalizare t5
24            if (t5_c == 1) { nP2++; P2[nP2] = t5_s; }
25            else { nP5++; P5[nP5] = t5_s; }
26            t5_c = 0; D[5] = -1;
27            break;
28        case 6: //finalizare t6
29            if (t6_c == 1) { STr[1] += ceas - t6_s; CT[1]++; }
30            else {
31                nP6++; P6_s[nP6] = t6_s; P6_c[nP6] = 3;
32                if (nP6 > maxP6) maxP6 = nP6;
33            }
34            t6_c = 0; D[6] = -1;
35            break;
36        case 7: //finalizare t7
37            if (t7_c == 2) { STr[2] += ceas - t7_s; CT[2]++; }
38            else {
39                nP6++; P6_s[nP6] = t7_s; P6_c[nP6] = 4;
40                if (nP6 > maxP6) maxP6 = nP6;
41            }
42            t7_c = 0; D[7] = -1;
43            break;
44        case 9: t9 = 1; D[9] = -1; break;
45        case 10: t10 = 1; D[10] = -1; break;
46    }
47 }

```